

FINAL YEAR PROJECT 2024-2025

# Ukulele Robot with Mobile Application

Chiu Wing Tung Crystal, 1155174499

Cheng Ka Chi, 1155175548

# Ukulele Robot with Mobile Application

## **Abstract**

The project aims to create an ukulele robot that can be controlled through a mobile application. The robot will be able to play the note or chord remotely. The application can also act as an accompaniment, to let the user have a real-time interactive experience.

## **Index Terms**

Ukulele, Robot, Mobile Application

## CONTENTS

|            |                                            |           |
|------------|--------------------------------------------|-----------|
| <b>I</b>   | <b>Introduction</b>                        | <b>4</b>  |
| I-A        | Background and Motivation . . . . .        | 4         |
| I-B        | Project Objectives . . . . .               | 4         |
| <b>II</b>  | <b>Literature Review</b>                   | <b>6</b>  |
| II-A       | Robotics in Music Performance . . . . .    | 6         |
| II-B       | Limitations in Existing Projects . . . . . | 6         |
| <b>III</b> | <b>System Overview</b>                     | <b>8</b>  |
| III-A      | Architecture Overview . . . . .            | 8         |
| III-B      | Sequence Diagram . . . . .                 | 9         |
| III-C      | Flow Chart . . . . .                       | 10        |
| III-D      | Functional Requirements . . . . .          | 11        |
| <b>IV</b>  | <b>System Design</b>                       | <b>13</b> |
| IV-A       | Hardware Design . . . . .                  | 13        |
| IV-B       | Software System Design . . . . .           | 13        |
| IV-C       | Communication Protocol Design . . . . .    | 14        |
| <b>V</b>   | <b>Hardware</b>                            | <b>17</b> |
| V-A        | Wiring . . . . .                           | 17        |
| V-B        | Component Selection . . . . .              | 19        |
| V-C        | 3D Printing . . . . .                      | 25        |
| <b>VI</b>  | <b>Software</b>                            | <b>38</b> |
| VI-A       | Mobile Application . . . . .               | 38        |
| VI-A1      | Introduction . . . . .                     | 38        |
| VI-A2      | Stack Overview . . . . .                   | 38        |
| VI-A3      | UI Design . . . . .                        | 39        |
| VI-A4      | Architecture . . . . .                     | 47        |
| VI-A5      | BLE Logic . . . . .                        | 47        |
| VI-A6      | Deployment . . . . .                       | 50        |

|             |                                                |           |
|-------------|------------------------------------------------|-----------|
| VI-B        | Raspberry Pi . . . . .                         | 51        |
| VI-B1       | Introduction . . . . .                         | 51        |
| VI-B2       | Workflow . . . . .                             | 51        |
| VI-B3       | GATT server . . . . .                          | 51        |
| VI-B4       | Command Parsing . . . . .                      | 53        |
| VI-B5       | Chord Mapping . . . . .                        | 54        |
| VI-B6       | Score Parsing and Playback Scheduler . . . . . | 54        |
| VI-B7       | Deployment . . . . .                           | 54        |
| VI-C        | Arduino . . . . .                              | 54        |
| VI-C1       | Introduction . . . . .                         | 54        |
| VI-C2       | Component Class . . . . .                      | 55        |
| VI-C3       | Config . . . . .                               | 56        |
| VI-C4       | Finite State Machine . . . . .                 | 56        |
| VI-C5       | Workflow . . . . .                             | 57        |
| <b>VII</b>  | <b>Division of Labor</b>                       | <b>60</b> |
| VII-A       | Detail Work . . . . .                          | 60        |
| VII-B       | Personal Contribution . . . . .                | 60        |
| <b>VIII</b> | <b>Conclusion</b>                              | <b>62</b> |
| VIII-A      | Summary . . . . .                              | 62        |
| VIII-B      | Future Work . . . . .                          | 62        |
|             | <b>References</b>                              | <b>64</b> |
| <b>IX</b>   | <b>Appendix</b>                                | <b>65</b> |
| IX-A        | Application Download Link . . . . .            | 65        |

## I. INTRODUCTION

### *A. Background and Motivation*

When learning a string instrument as a beginner, such as guitar and ukulele, it is common to search for different chords, and sometimes it may take a long time to find all the necessary chords or fingering styling. On one hand, some beginners would like a demo of how the song is played as a reference to verify their own understanding and performance. [2] On the other hand, some ukulele lovers may want to listen to the sound track of a song you are interested in but unable to search it online and listen to it right away, or you would like to perform a duet but you have no partner. To address the aforementioned problems, we would like to propose with a view to merge technology and music by providing solutions to both music lovers and music learners.

### *B. Project Objectives*

The merging of a robotic system with a mobile app focuses on increasing engagement when playing the ukulele while also providing a solution to the growing demand for automated Instruments. The ability to play difficult chords and techniques means that the ukulele robot allow the average user to enjoy the experiences of live performances without the performer having to be particularly skilled; this makes music easy and accessible even for novice players. Moreover, this project also addresses the issue of music education as it provides an interesting instrument for practice and learning with which the users can learn various songs and styles at their own pace. All in all, this initiative promotes creativity and discovery into music, inculcating the users with a broader sense of understanding and appreciation of musical arts and the use of technology to achieve this.

This report outlines the approach of making a ukulele robot with an accompanying mobile application to create more engaging interactions with music. The main focus of the project is to obtain a robotic system, which is fully autonomous that can play a ukulele and enable the users to pick a song and change the chords in real-time through controlling. The hardware includes a multi-arm mechanism to preform string pressing and servo motors to wiggle the strings by mimicking various fingering techniques. The mobile app allows the users to select songs to play

with, therefore improving the overall experience of the user. Early tests confirm that almost all chords were played by the robot with hard strokes but little sound which is one of the problems existing in all robotic musical instruments. This project is not only related to robotic musical performances but also probes new ways for making music and interaction with the audience.

## II. LITERATURE REVIEW

### *A. Robotics in Music Performance*

Integrating robotics into the music industry has become increasingly common in recent years. A robotic musical instrument is characterized as a device that autonomously generates music through the utilization of mechanical components, including motors, gears, and solenoids [3]. With the incorporation of technology, this facilitates performance, accessibility, and even composition of music. The convergence of robotics and musical art has led to the development of diverse types of musical robots, including piano robots, turntable robots, percussion robots, string robots, and wind robots [3].

After studying the latest trends in the area of string robots, especially the guitar and ukulele robots, it was found that although there are many recorded resources on guitar projects, there is less documented information on ukulele robots, making it challenging for users to learn about them. Additionally, most of the current ukulele projects incorporate solenoid mechanisms which attract and release a magnet for string pressing[4].

### *B. Limitations in Existing Projects*

After reviewing multiple solution online, it has been observed that the approach of using solenoid will result in creating significant noise that affect the user experience as it interrupted the musical performance [5]. While a minor of current project address such unwanted sound from happening by replacing each solenoid with servomotor and a arm structure to press the string. However, the approach of this project only focus on providing limited pressing of the chord as it only install the servomotor in few frets but not allowing user to access and press all the chord they can also the sound of tuning gear can also be significantly noticed[6].

While the above research project are all using computer to directly control the robot, it has been find out another project that integrate a screen with the robot to enable control of the robot directly without bring computer all along [7].

After understand the different approach of the current project in the market and their pros and cons, we would like build a project that can provide better user experience in terms of quality of the music, user-friendliness and convenience.

### III. SYSTEM OVERVIEW

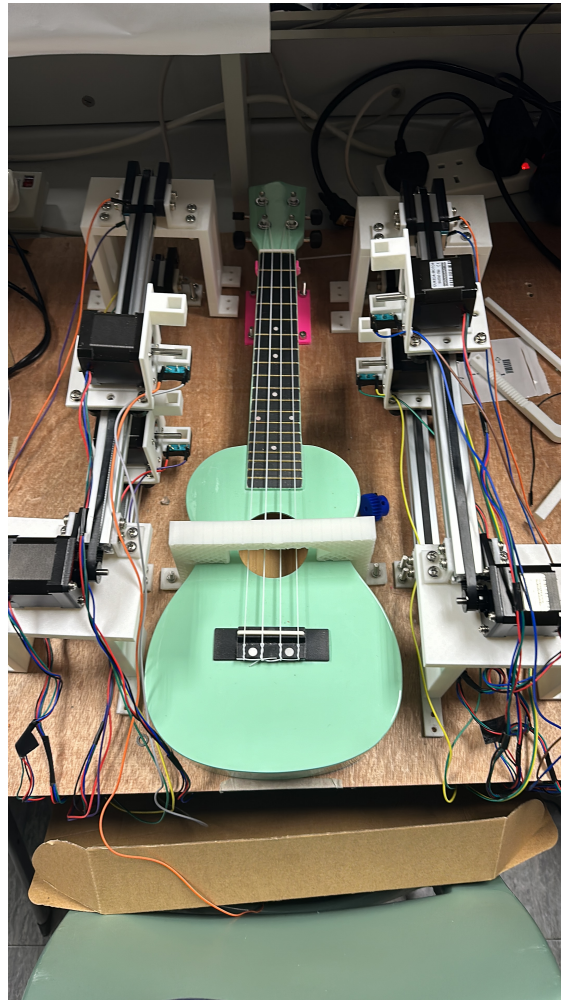


Fig. 1. Top view of the ukulele robot.

#### *A. Architecture Overview*

This project is composed of the collaboration of both hardware and software.

The hardware part includes multiple parts to simulate the human interaction with the ukulele. First, the robot is equipped with multiple arms. The arms are connected to a slider to control the vertical move and an extra motor for horizontal move. The arms will press chords and perform different fingering techniques. This design is to simulate the human playing.

The software is an additional to the hardware component. The mobile application will allow the user to choose their own songs. It can also let the user play a specific chord or fingering

technique. It can provide an interface for users to use the self-playing function of the robot. Integrating with software, the robot can improve the performance of music playing and provide a personalized music experience.

Here are our system design diagrams:

### B. Sequence Diagram

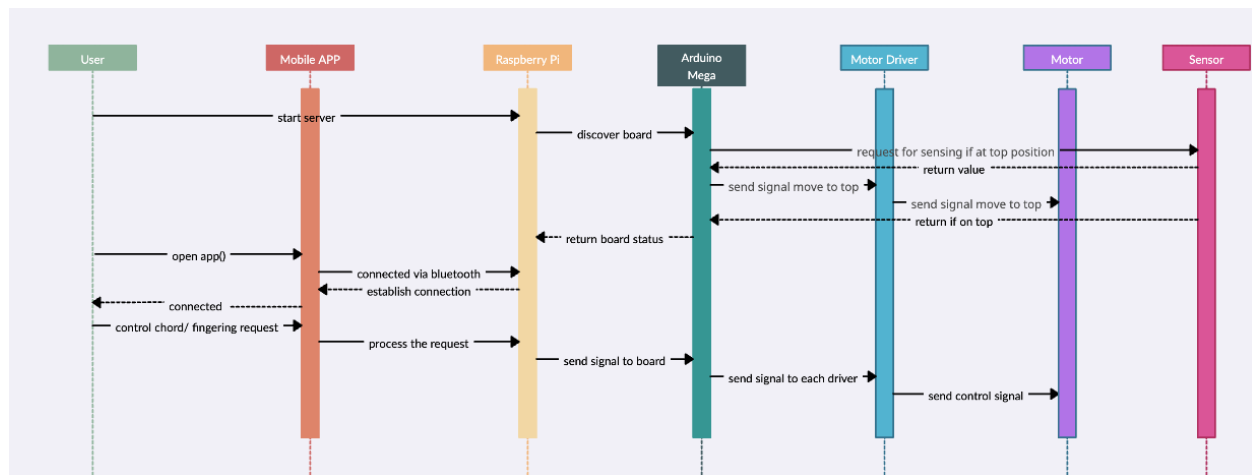


Fig. 2. Sequence diagram of the flow of whole system

### Step-By-Step Descriptions:

- 1) Once the server started, the mechanical arms will be requested to reset to initial position by Mega.
- 2) It will check with the sensor at the initial position to see if the arms are of right position.
- 3) If it is reported that the arms are not in the initial position, Mega will send signal to each driver to request them to move to top.
- 4) Mega will send signal to Raspberry Pi notify the reset setup is done to let it broadcast the Bluetooth LE Signal.
- 5) The user open the app.
- 6) The app would receive the Bluetooth LE signal from robot.
- 7) The mobile app would request to connected with Raspberry Pi via Bluetooth.
- 8) Raspberry Pi establish connection with the mobile app via Bluetooth.

- 9) User input the chord they want to play in the mobile app.
- 10) Mobile app send chord request or command to Raspberry Pi.
- 11) Raspberry Pi process the chord request send signal to Mega.
- 12) Mega after receiving request will send signal to corresponding motor and preform movement.

### C. Flow Chart

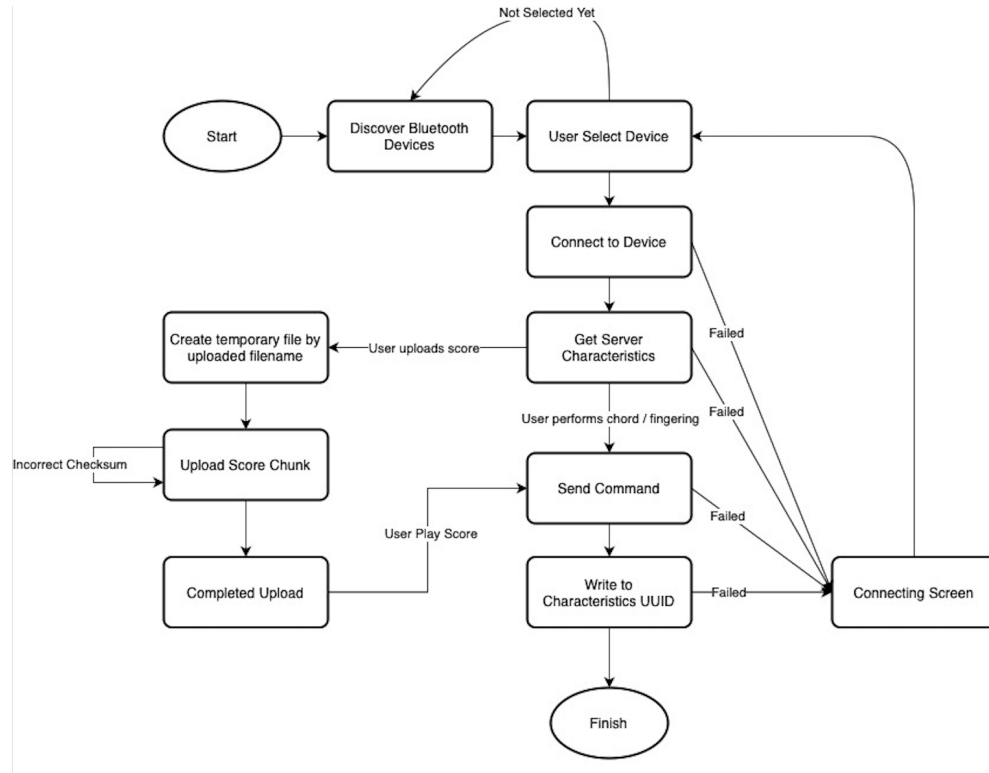


Fig. 3. Flowchart of Mobile Application.

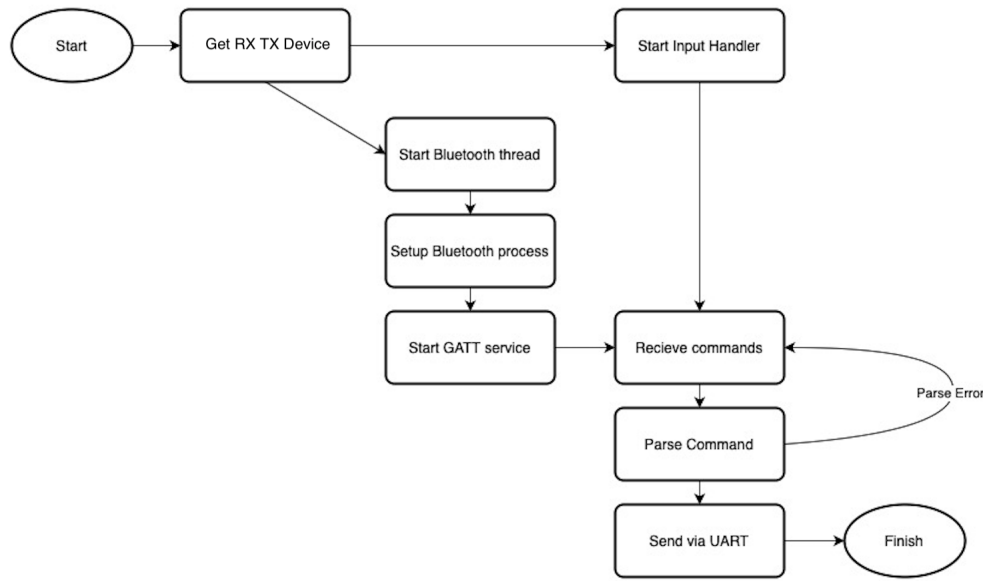


Fig. 4. Flowchart of Raspberry Pi.

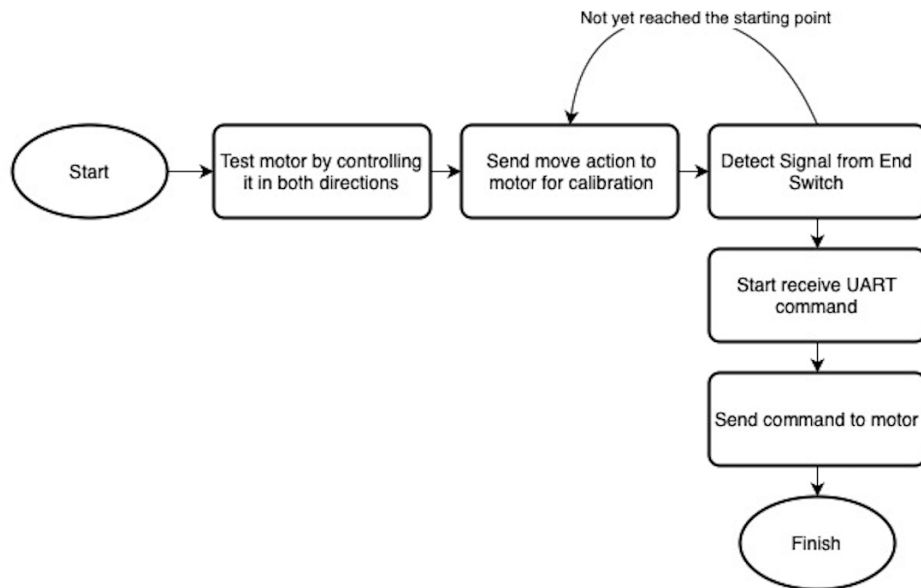


Fig. 5. Flowchart of Arduino Mega.

#### D. Functional Requirements

The ukulele robot should be equipped with the following features:

### 1) Play Fingering

The user should be able to perform fingering on the application. They should be able to choose the specific fret and string, after that the robot will play that note simultaneously. The robot should react to the request instantly.

### 2) Play Chord

The user should be able to select a specific chord, after that the robot should move four fingering unit together and perform a chord.

### 3) Play Score

The user should be able to upload the ukulele score to the robot. Currently, the major file format like MusicXML and MIDI are supported. After that, the user can choose to play the score and it will start to move the fingering units to achieve this goal.

To achieve the above actions, several software and hardware components are involved. For example, here are one of the scenarios where the user plays the score.

#### 1) The user uploads the score.

The mobile app will retrieve the file from the user device. After that, it will send the file to the Raspberry Pi using BLE. Multiple characteristics are designed for different actions.

#### 2) The user chooses to play the score.

After the user chooses the file and performs the play action, the app will send the action to the Raspberry Pi. After that, the Raspberry Pi will start to parse the score and transform it to an intermediate code. Those commands will be sent to the Arduino board based on the time that the score specifies.

#### 3) The robot plays the note.

After the code is sent to the Arduino board, it will start to move the fingering unit by sending the digital signal to different pins. After that, the motor will start moving and play different notes.

## IV. SYSTEM DESIGN

### A. *Hardware Design*

The hardware component consists of the wiring, electronic components, and 3D printing. According to the experiment carried out in previous semester, a one-layer design requires a long y-axis design of the rack and may result in inadequate strength at an extended distance and reduced pressing accuracy due to angular vibration. In light of it, the ukulele robot adopts a two-layer slider stacked design. In such circumstance, the rack is extended on the z-axis, that is, the vertical distance with the string, while minimizing the x-axis, which means the horizontal distance along the width of the rack to the string, thus improving stability and maintaining accuracy.

For wiring, as this project includes a bunch of electronic components, it is essential to organize the wiring between the components, the Arduino Mega, and the power source to ensure that each component can work smoothly.

The key to this project is the ability to play the ukulele which consists of two parts, upper part of the robot to perform fingering on the fret-board, and the lower part of the robot to perform strumming or plucking of strings. By choosing suitable electronic components along with designing 3D printing to tailor based on our purpose.

To ensure consistency in performance, the ukulele, those 3D printed parts and electronic parts will be mounted on a wooden board for position fixing. This enhances the effective mechanical interaction and ensures that there are no disturbances while performing. Due to the defined placement of these constituents, precision of the chord pressing and fingering can be achieved.

### B. *Software System Design*

The software component consists of three parts: the Arduino Mega, Raspberry Pi, and the mobile application.

For the Arduino Mega, it is used to control the motors. The Mega board is connected using an UART interface. The Mega board is controlling the vertical and horizontal movement of the

rack, also it is used to pluck the strings.

The Raspberry Pi is used to receive Bluetooth signals from the mobile device and relay commands to the Arduino Mega boards. It acts as a bridge between the user interface and the hardware components.

For the mobile application, we are using a cross-platform framework. React Native is currently chosen, as it has a large community and supports the simultaneous development of both Android and iOS applications.

### *C. Communication Protocol Design*

There are two types of protocol designed for the project. One is for the communication of Arduino board and Raspberry Pi through the UART(TX/RX) ports. Another is the connection between the client device and the Raspberry Pi through Bluetooth LE connection.

The protocol is as follows.

#### 1) UART communication

In previous stage of our project, we use I2C as a communication method between Arduino board and Raspberry Pi. The command is sent on byte based.

|             |   |        |          |   |          |   |   |   |           |
|-------------|---|--------|----------|---|----------|---|---|---|-----------|
|             | 1 | 2      | 3        | 4 | 5        | 6 | 7 | 8 | 9         |
| ”Control”   | 0 | Target | Speed    |   | Duration |   |   |   | Direction |
| ”Calibrate” | 1 | Target |          |   |          |   |   |   |           |
| ”Move”      | 2 | Target | Distance |   |          |   |   |   |           |

As the program become larger, our command needs to handle more items like Motor ID. Also, there are a limitation on range of parameter due to the limited number of bytes. Hence, the current byte array is not suitable for the communication. Besides, we have reduce the number of Arduino to a single board in the current stage. Hence, UART is a

more suitable and stable method.

Currently, our program using the below format.

|             |                                                     |
|-------------|-----------------------------------------------------|
| Control     | S [motorID] [target] [speed] [direction] [duration] |
| Calibration | C [motorID] [target]                                |
| Move        | M [motorID] [target] [distance]                     |
| Debug       | D [motorID] [target] [position]                     |

Our command will be sent as string format. As UART has connection checking, it will throw IO error if the connection is failed. Hence, no byte checking is needed.

The Arduino side will keep checking if there are bytes sending from Raspberry Pi. If there are, it will save the received string to the buffer. After the newline character is recieved, it will start process the command.

## 2) BLE connection

To send signal using Bluetooth, we would have two approaches, which is GATT server and Serial to transmit data. However, due to the limitation on iOS platform, we can only use GATT as a choice. The issue will be addressed in later paragraph.

GATT is the core part of data transmitting. If GATT service is not enabled, the client side can only pair to the device but cannot transmit data. However, there are no GATT packages for python. We used the dbus interface to interact with the Bluetooth process. After the process is connected, we have attached multiple services into an Application as a GATT service. After that, the server will advertise its service and characteristic. The service will get value from the client and pass it to the user input part to parse the command.

Here are the services list. Details will explained in later chapters.

| Service             | Characteristic | Read | Write | Description                                |
|---------------------|----------------|------|-------|--------------------------------------------|
| MotorService        | MotorWriteChrc | ✓    | ✓     | Accept actions to motor and return success |
| PlayAudioService    | ListFilesChrc  | ✓    |       | Return the list of saved file              |
| PlayAudioService    | PlayAudioChrc  |      | ✓     | Play the selected file                     |
| PlayAudioService    | PauseAudioChrc |      | ✓     | Pause the playing file                     |
| PlayAudioService    | DeleteFileChrc |      | ✓     | Delete the selected file                   |
| FileTransferService | FileWriteChrc  | ✓    | ✓     | Accept stream upload and return checksum   |

## V. HARDWARE

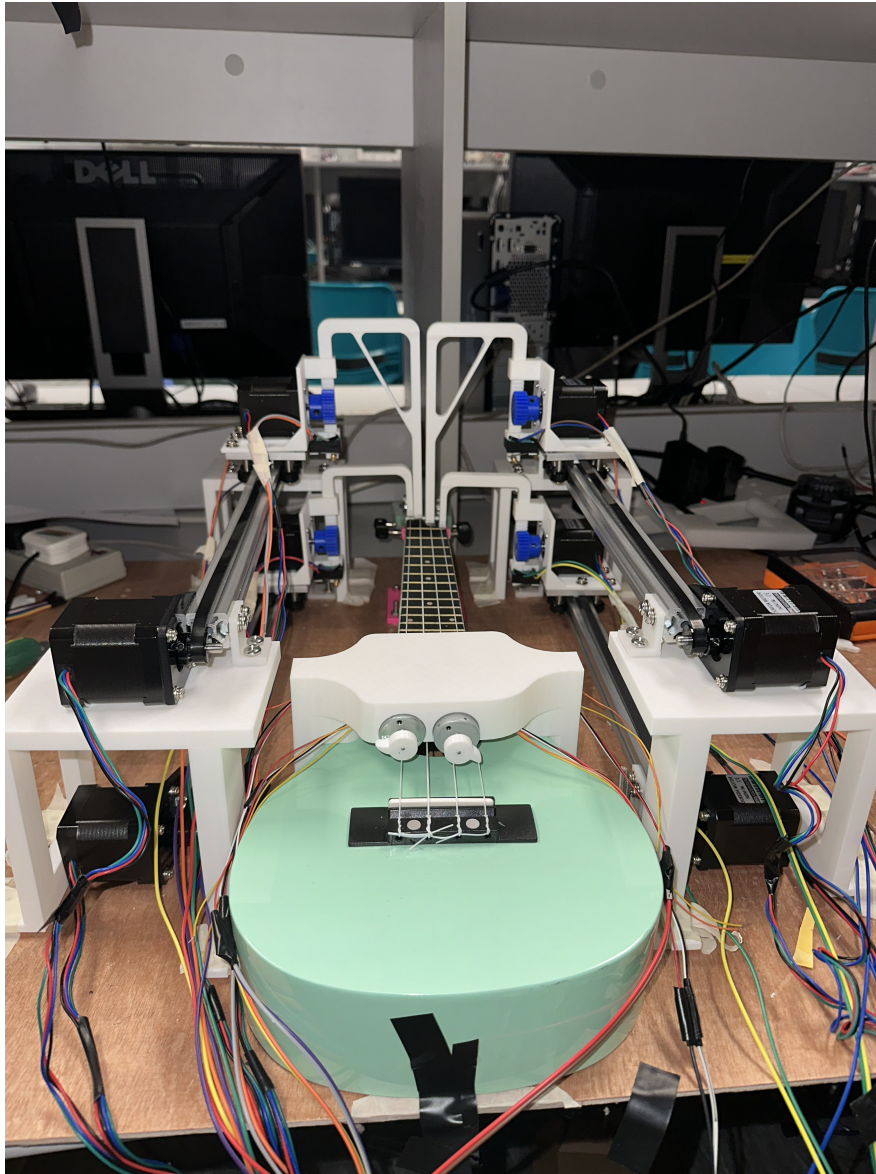


Fig. 6. Front view of the ukulele robot.

### A. Wiring

To provide synchronization among different electronic components, we have to connect all of it to one major board for coding, in this project Arduino Mega is used.

Currently, each slider and stepper motor are connected to their corresponding driver, and both are wired to the Arduino Mega for over-all communication. Without driver, insufficient power is supplied to the motor, therefore, to provide enough power for rotation. Meanwhile, an end

switch is connected to each slider and rack for calibration. There are three pins, namely, common terminal, normal open, normal close. The common terminal is connected to the 5V power supply, while the normal close other one for the signal output and ground through  $10k\Omega$  resistor. The reason for connecting to the resistor as an external pull-down is to prevent the analog input from floating when not pressed or to drive the voltage low enough quickly hence delay in signal or false reading.

In this project there are two sets of separate power sources, namely 24V and 5V. All NEMA 17 motors for the rack and slider require a 24V power supply through connecting through each motor driver; while all other electronic components, in particular, Raspberry Pi, Arduino Mega, DC motor through Relay, end switch are all using 5V power supply. In this case, for simplicity and better organization we connect all 24V channels and power sources in parallel, and the same applied to the 5V power supply. With a large number of components connecting together with high voltage, we connect both power supply with isolation transformers to protect the device testing and measurement.

The TX RX port of the Arduino Mega is then connected to the Raspberry Pi's TX RX for communication among the mobile app and the ukulele robot. The ground of both boards are connecting together and ultimately to the 5V ground.

The following is the wiring diagram showing how one set of arm and pluck is connected to the Raspberry Pi and Arduino Mega:

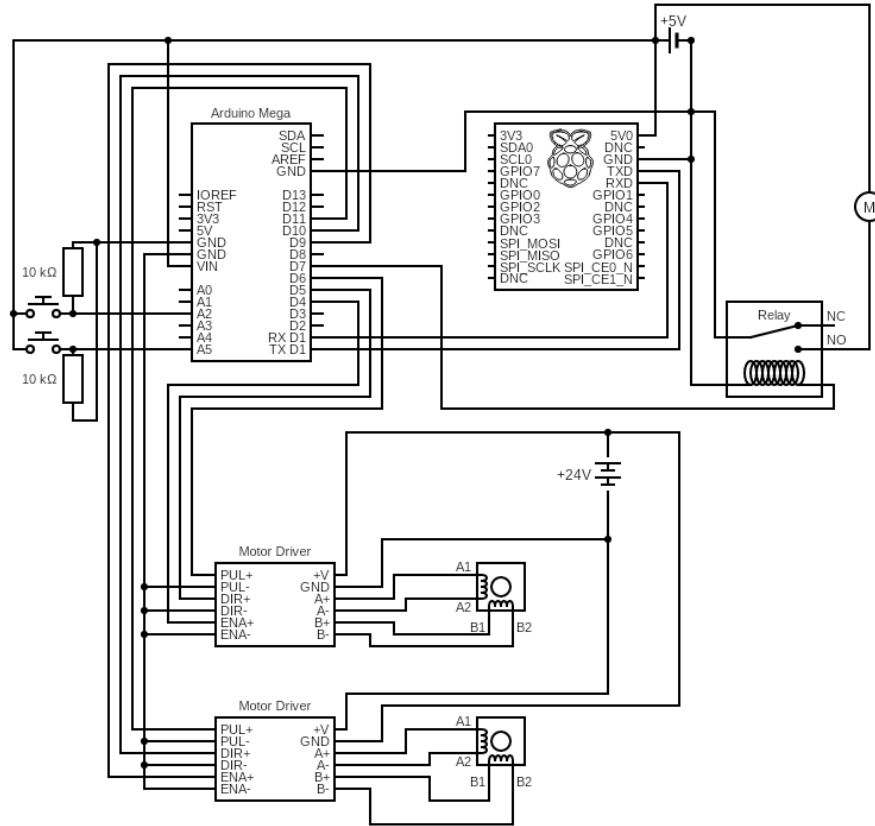


Fig. 7. This graph showcase how the wiring among one set of arms is inter-connected to a Mega board hence a Raspberry Pi. And ultimately there will be four set of motor drivers connecting to the same Mega board.

### B. Component Selection

In this project, different electronic components are embedded in the system to help put the idea into practice. The type of electronic components must be chosen wisely to provide the expected functionality. There are four main parts we have to consider, namely stepper motor, slider, DC motor, and end switch.

The common approach for such string-pressing functionality is performed using solenoids placed on top of every intersection of the frets and strings, pressing the string by releasing the armature onto it to apply pressure, mimicking the pressing action of a human finger. This approach requires a large number of solenoids, and hence a significant amount of wire for connection [8]. Another crucial issue mentioned in the literature review is the unavoidable

clicking noise when the armature is released by the solenoid.

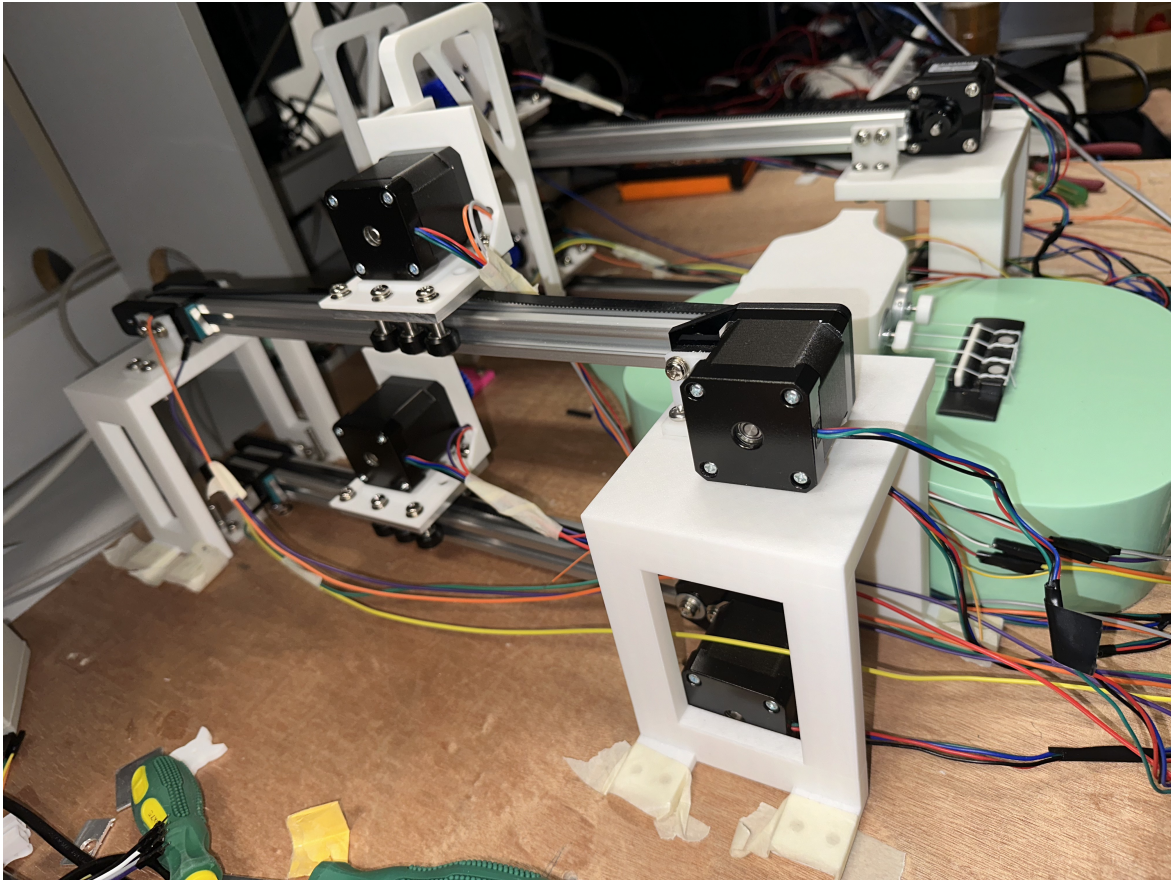


Fig. 8. Side view of the ukulele robot showcasing the structure of the gear and rack.

To prevent such noise disruptions that affect user experience and enjoyment, our project decided to apply a stepper motor incorporated with a gear and rack to manipulate the vertical action of string pressing. Stepper motors allow for precise control of the angle of rotation, and thus the movement of the rack.

There are typically three types of stepper motor namely, permanent magnet stepper motor, variable reluctance stepper motor and hybrid stepper motor, which is the combination of the previous two. The permanent magnet stepper motor makes use of the permanent magnet to control the angular movement by creating two magnetic fields to interact with each other providing high torque while maintaining low speed. Variable reluctance stepper motor makes use of the interaction between magnetic fields and the soft iron rotor to perform rotation offering comparatively

low cogging torque. Where hybrid stepper motors combine both structures. resulting in the benefit of the aforementioned stepper motor[9]. Therefore, a hybrid stepper motor has been chosen for the project.

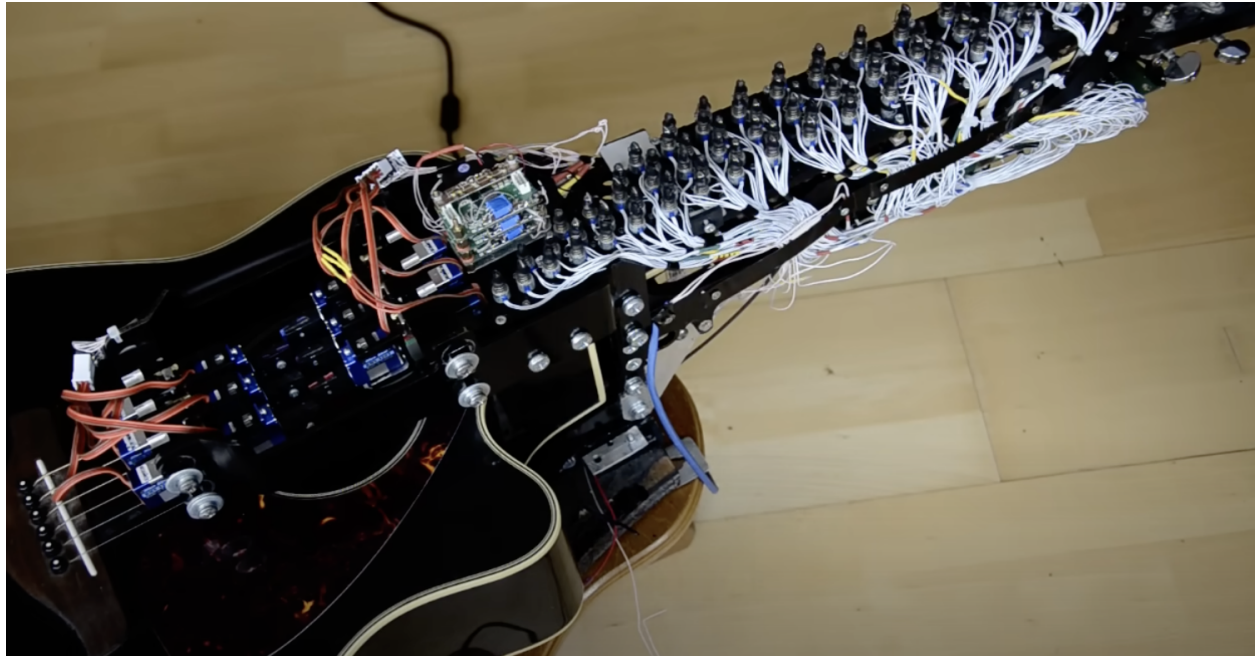


Fig. 9. Common practice of using solenoids with a bunch of wire needed.

Among the different types of hybrid stepper motors, NEMA 17 stepper motors are famous for their high torque and reliability, which match the objectives of our project. While the stepper motor only supports the rotation, a mechanism is needed to design a transform from rotation to vertical movement, which a gear installed onto the shaft with a rack that will move correspondingly when the gear is rotated. On top of each slider plate, a NEMA 17 stepper motor is placed with a geared rack, providing vertical movement along the z-axis, to ensure that the robot can form proper chord shapes on the strings.

In order to prevent the slider and the rack from rotating without knowing when to stop and collide with other parts is cause a huge noise and wear out over time, therefore a sensor is needed for limit detection. One of the common practices of sensor control is the infrared sensor, which detects the signal of an object through infrared reflection, which does not require

physical contact. However, performance might be affected by environmental factors, for instance, surrounding light, dust, and the color of the reflected surface; hence, it may result in inconsistent results and delay. In spite of the high precision and consistency of the positioning of the slider plate and rack is needed to calculate the distance of different frets, delay of the signal or false trigger is prohibited.

Another approach is to use mechanical end switch, to provide a signal when object is physically intact with the end switch. As such, immediate and reliable signal transmission will be sent back to the Arduino Mega with short latency. Therefore, the end switch is widely used in this project to detect if the slider plate or the rack reaches its top position.

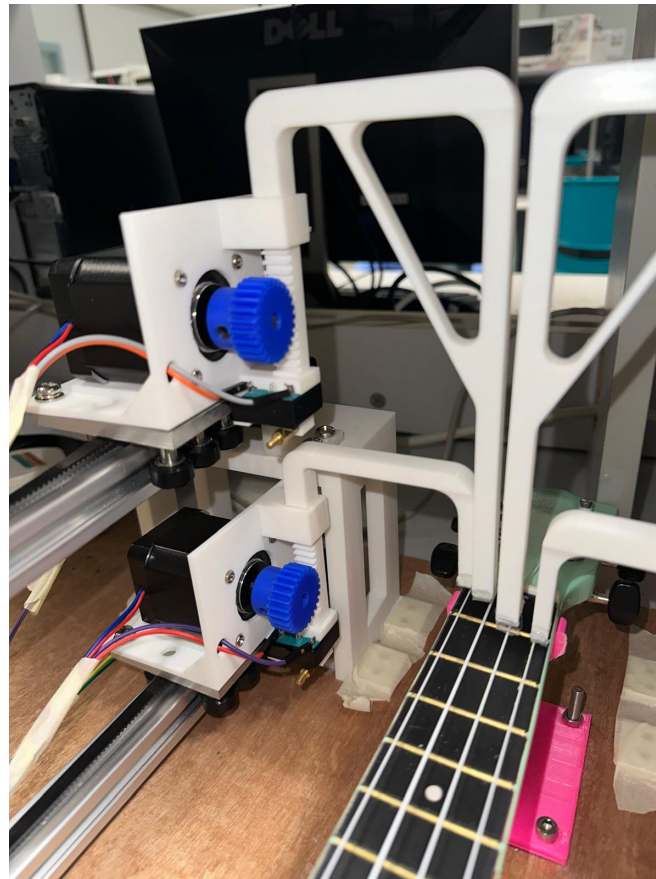


Fig. 10. End switches are placed in the on the side of the bottom of the tunnel to test if the rack reaches it's top position.

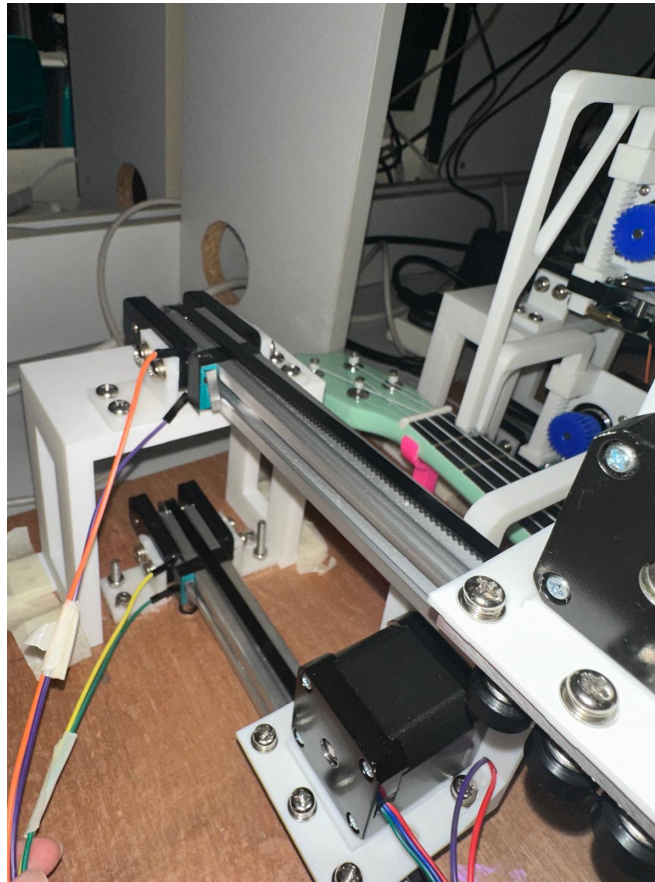


Fig. 11. End switches are placed in the on the end of the slider to test if the rack reaches it's start position.

After handling the vertical movement of the arm to locate the precise position of each fret, the next concern is the implementation of the horizontal movement along the depth of the arm.

The reciprocating 42-step slider has been used to feature belt transmission. This physical characteristic of the slider, which allows for linear movement within a short period of time, is important to ensuring that there is no unnecessary motion of the slider due to vibrations. Preventing deviation of the slider ensures consistent operation. However, the 42-step motor came along with the slider prone to overheat and melt the plastic holder nearby. With safety concerns, the motor is then changed to the NEMA 17 stepper motor, which is used for the rack due to its high thermal stability and seamless and quiet motion.

To facilitate the calibration of the system, the end switch is positioned at the bottom of the slider plate to check if the rack reaches its highest position, which is also the initial position;

similarly, it is also placed at one end of the slider, which is opposite to the stepper motor to see if it reaches the end.

NEMA 17 motor driver is used to work as a bridge to send signal from Arduino Mega to control the stepper motor. In the meantime, the motor driver can also be adjusted to tailor the different pulses and currents used. The robot is using the following setting. Which S1 - S3 is controlling the pulse, in this case is at micro step 2/B and 400 pulse per sec, meaning one full revolution per second; S4 - S6 indicates the current and peak current setting, in this case 2.5 A and 2.7 A respectively. Under this setup, the reduces the mechanical vibration and hence minimizes operational noise. Also, setting the peak current under the maximum current it can handle 2.8A, allowing the operation of the motor smoothly without overheating.

|    |     |
|----|-----|
| S1 | OFF |
| S2 | ON  |
| S3 | ON  |
| S4 | OFF |
| S5 | ON  |
| S6 | ON  |

TABLE I

THE PULSE AND CURRENT MODE OF THE MOTOR DRIVE CHOSEN FOR THE PROJECT.

Referencing to multiple ongoing projects in the market, it can be observed that most of the project is using a servo motor as the mechanism for plucking due to its ability to achieve accurate angle control and fit in tight space.

Although a servo motor is a popular option, those who can rotate in 360 degrees are too large, and the turning sound of the gear will also be apparent. As a result, the DC motor is considered in this project, as it provides higher rotational speed and stronger torque. In addition, it is noise-free during operation, allowing the robot to provide better music quality.

Each DC motor is connected to the Arduino Mega through a relay for several reasons. First, to perform electrical isolation between the motor and the logic circuit, Arduino Mega, to avoid

breakdown of the logic circuit due to single-motor failure. Additionally, we control the frequency using on and off, which relay can help control with it. Even though there maybe a clicking sound when turning on and off the relay, yet it is placed in a box under the robot and unlike the noise is not continuous during operation.

### *C. 3D Printing*

Within this project, it is necessary to design different components that cater to our needs in terms of functionality and size, for instance, the design of the rack and holder, etc. Among the different options, 3D printing technology has been chosen, as it can be printed according to our design, providing precision in the required shape and size. The two commonly used materials for prototyping fused deposition modeling (FDM) 3D printing are Acrylonitrile Butadiene Styrene (ABS) and Polylactic Acid (PLA). In the project, it is crucial to have 3D print surface precise and smooth, minimizes noise and allows different parts to move seamlessly. Therefore, PLA has been chosen due to its higher tensile strength and smoother surface finish [10] .

SOLIDWORK 2024 is being used as a tool to design all 3D components. In this project, we have to design and assemble parts with different electronic components we used, the assemble parts that cannot be found easily found in the market. To deal with it, designing 3D components tailored to our need is essential to assemble the product. With the feature of designing it component by component, it allows us to assemble different parts to have an overall picture of the product and demo movement before printing. This allows problem to be observed and corrective amendment made before printing, reducing the time to amend the draft due to the inability to assemble after printing all components.

To print out the 3D component, OrcaSlicer is used over Bambu Studio due to more advanced built-in calibration tools and more precise fine-tuning of the setting of the 3D print to set least brim as possible and ensure the printed parts are exact the same size as what we designed.

In this project, multiple parts are made with 3D printing, namely the slider stabilizer, ukulele stand, ukulele holder, and a holder placed on the slider to position the rack and motor, ukulele

pluck, rack.

As mentioned, a multi-arm mechanism is tailored to perform the pressing of the strings. There are four sets of arms in total to handle the four corresponding strings. The arm mechanism is composed of two parts: a slider controlling the horizontal movement along the depth, y-axis, to reach different frets and a rack with a gear controlled by a stepper motor to perform the vertical movement along the z-axis to press different strings.

The holder is designed to be placed on top of the slider to hold the motor driver, gear, and rack in place. One of the key features of the holder is to leave room for the rack to move while ensuring that the space is not too large, preventing the rack from wobbling during vertical movement. The base of the holder is drawn the same as the slider plate, with nine screw holes to allow M5 screws to be installed, ensuring that the holder is tightly secured during movement. Above the base, there is a platform designed to house our NEMA 17 motor driver. Without the platform, the screw hole will be covered by the tunnel. Not installing all screws might result in an error in position calculation and unwanted wobbling. A leveled platform ensures that there is no height difference between the motor and the holder, allowing the screws to be installed and confirming that it is leveled. From the front view, a hole is reserved for the shaft to pass through. The four screw holes in the front view align with the front endcap of the motor, allowing M3 screws to be installed for positioning.

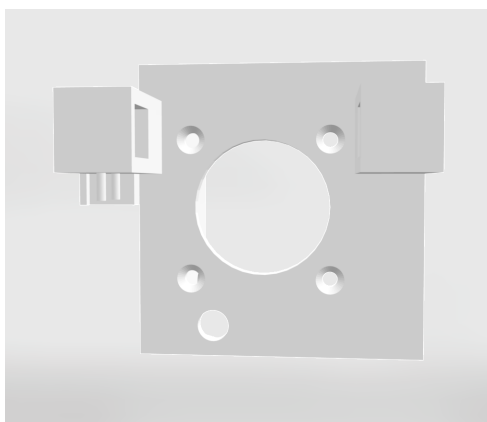


Fig. 12. Front view of holder with four M3 screws.



Fig. 13. Side view of holder with platform and two square tunnel featured and a small triangular support to prevent deformation due to the torque of pressing string.

Another feature is the two square tunnels on the side of the holder, which are for placing the rack and guaranteeing that the movement of the rack aligns vertically. Without the tunnel on the side, the rack will not be able to place in the same position and not able to move vertically solely depending of the gear as it may disassemble due to gravity. The entrance of the tunnel is adjusted to be slightly larger than the rack, leaving only minimum space for it to pass through the tunnel and move vertically. Leaving a small gap for movement will still result a strange sound due to wiggling. This problem by adjusting the 3D print to leave minimal space for movement and utilize the action-reaction force applied by the gear in stabilizing the rack. The position of the tunnel is chosen based on the following concern. If the tunnel is too long, it may touches the wooden board and may add constraint to the height of the rack to be higher in order not to clash with the holder. Therefore, we measured the difference of the holder with the board when deciding the length of the lower tunnel and set the upper tunnel length not to exceed the total height of the holder, hence minimize the limitation in affecting the design of the rack.

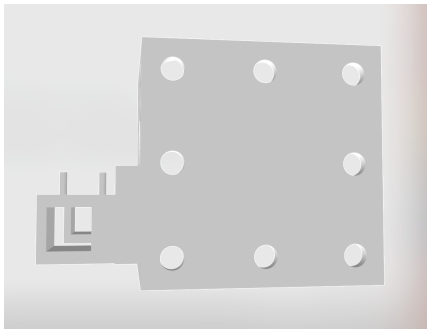


Fig. 14. Top view of holder showcasing the entrance of the tunnel and screw holes of the holder for install screw on the slider.

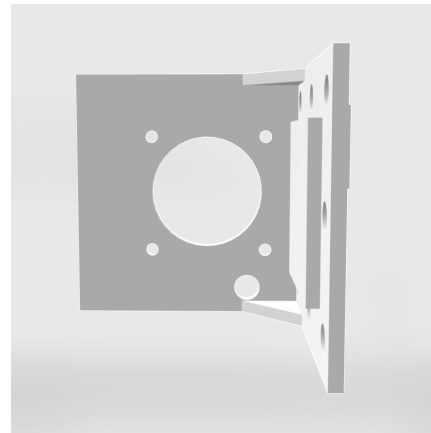


Fig. 15. Back view of holder showing the extended part of the tunnel.

After concerning the height and size of each tunnel, the position of the tunnel carefully designed concerning the mechanism of using gear. First of all we choose the gear with 35 teeth and radius of 1 cm, which is the minimum radius with the maximum teeth we can get. The smaller the gear the less space it take up, while the more teeth the gear have indicating we can have precise control of the length we would like to have. With such an approach, the tunnel needs to be placed in a position that just fits the horizontal position of the rack when it assembles

with the gear and in a position that will not affect the installation of the motor screws on the holder.

In last semester, it has been observed that the holder may blend forward when the rack is being pressed downward. In the long term, it may result in deformation and accuracy of vertical distance measurement of the rack should go. To deal with it, a triangle barrier is added to both sides of the frame to provide support and prevent blending.

On the side of the lower tunnel, two sticks are designed to hold the end switch in place by plugging the end switch to the two sticks. The L shape block is to ensure that all end switch are on the same horizontal position.

Each slider plate will be paired with one holder, and the holders for the sliders on the left and right are flipped along the y-axis to match different sides.

The overview of holder draft and the close-up of end switch feature are provided below:



Fig. 16. Side view of the slider featuring the placing of end switch on the slider.

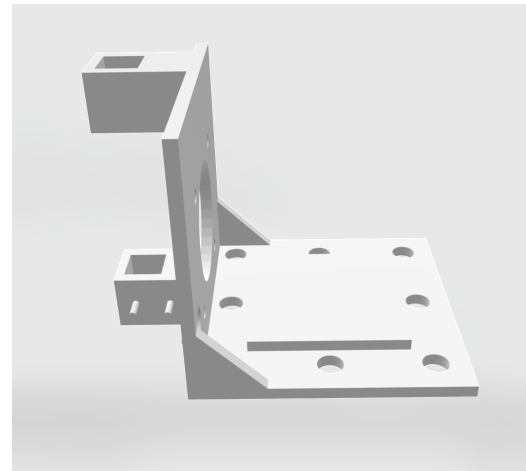


Fig. 17. 3D graph of the holder.

In order to press different strings, a special rack is tailored with two different lengths along the z-axis. The rack is designed to allow precise vertical movement together with the gear, with a series of teeth placed in the middle of the y-axis of the rack. The total length of the series of teeth is measured based on the range of the vertical movement needed for the rack to press the string without clashing. Considering a case that when the inner string is presses and outer

string is not press and require access to another frets. The range of the rack that press the in inner layer should not affect the movement of the outer rack, hence determine the length of the teeth of the rack.

The other reason for reserving a recessed area for placing the teeth, with this approach it will not exceed the entrance of the tunnel eliminate all possibility of easily getting stuck in the holder. The other part of the rack, not in contact with the gear, remains smooth to avoid unwanted instability that could occur if the teeth were to get stuck in the tunnel during movement. To press the string, a rectangular stick of similar width among strings is added to the tip of the z-axis component of the rack. And the whole design of the rack is to make it rounded for better distribution of force and prevent collision among rack.

This robot adopts a double layer design; hence the two racks that use to press the inner strings are designed to be of longer height and to press from the stacked up slider on the top layer. To prevent the string pressing stick from blending or vibrating due to insufficient support, a triangular support is added for longer racks.

The height and width of the long and short racks are precisely measured and calculated to prevent any clashing during the movement.

Another feature of the rack is that there is a small M3 screw hold designed for adding M3 x 12 screw with a M3 x 12 stand to press the end switch placed in the slider holder. Similar to slider holder, racks for left and right are flipped along the y-axis to match different sides.

The design draft is shown as follows:



Fig. 18. This is the rack designed to place at the right to press the outer right string.



Fig. 19. This is the rack designed to place at the right to press the inner right string.

To ensure the multi-arm mechanism can reach the correct position every time, it is necessary to fix both the ukulele and the sliders onto a wooden board. For the ukulele, a holder is designed to hold the body, and a stand is designed to support the neck. The holder is designed to hold the hourglass-shaped body of the ukulele at the thinnest part of the ukulele, near the sound-hole and leaving room for string. By holding the thinnest part of the ukulele can avoid ukulele to move back and forth, meaning the x-axis.

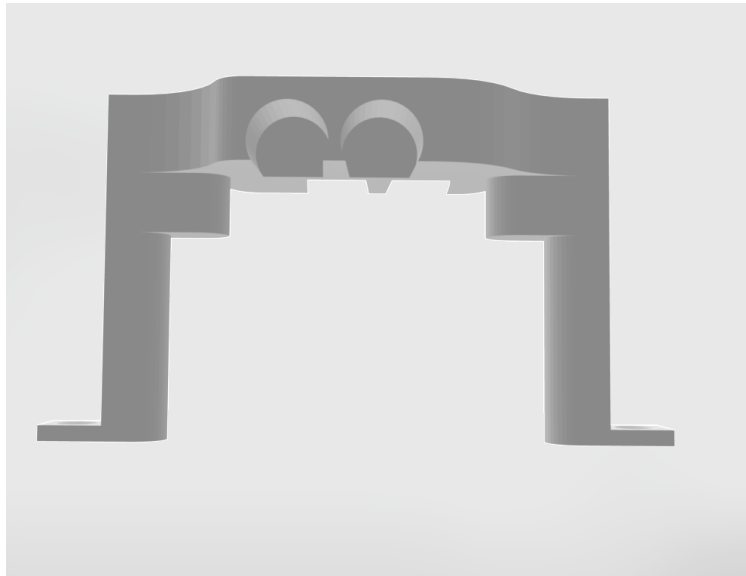


Fig. 20. Ukulele holder with upper room left for strings and DC motor.

Besides, this ukulele holder is designed to hold the DC motor in position to press for different strings. Due to the motor size limitation, the back and front side of the holder is each responsible to hold two motors in place for fingerpicking the alternate string, in detail, string 1 and 3 on one side; string 2 and 4 on the other side.

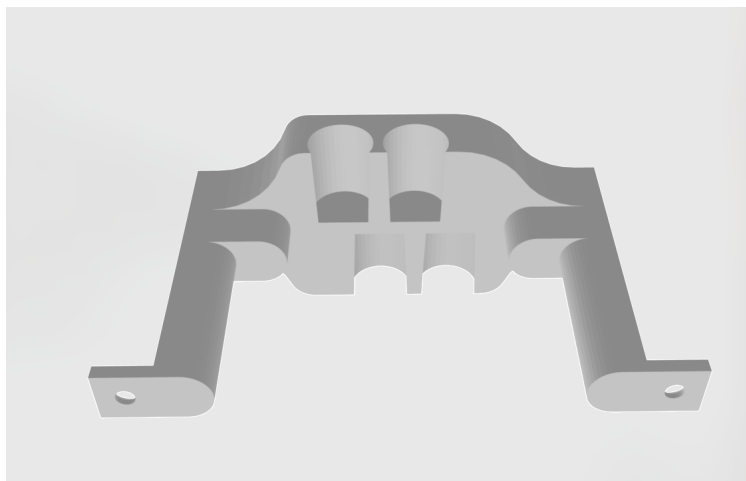


Fig. 21. Ukulele holder bottom view to see the layout of the motor holder pattern.

Moreover, when the string is pressed for chord or fingering, the string will be lowered and the pluck is unable to strum the string if it is placed leveled, if the pluck is designed to be longer

in diameter, they may clash with the nearby pluck or may harm the string due to great torque strength. Because the ukulele has inclined forward based on its own design, there is a height difference from the ukulele holder and the string of front and back views. To address the above obstacle, the holes designed to place the motor in the front and back view are of different slope to meet our requirement. To identify the front and back sides of the ukulele holder, there are slight differences in the top design of the ukulele holder.

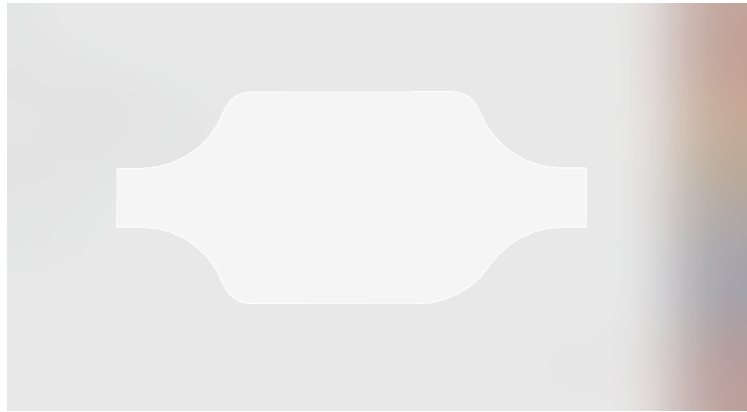


Fig. 22. Top view of the ukulele stand that the lower part with more curved surface is of a less steeper slope hole.

However, the ukulele may move from side to side if it is solely depend of the ukulele holder, to prevent such happened, a ukulele stand is designed to placed at the thinnest part of the neck to secure the movement along z-axis. With the holder pressing down on the body and the stand supporting the neck, a stable and level position is ensured, avoiding unwanted deviations. The design draft is attached below:



Fig. 23. Here is the front view of the ukulele stand.

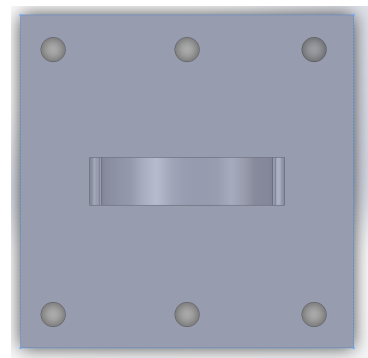


Fig. 24. Here is the top view of the ukulele stand.

To carry out string playing, the motor needed to be integrated with a plucking element. A gear-like pluck with one tooth is designed to press the string once on every command since the minimum rotation is one cycle. From the front view, it can be seen that the teeth have one rounded end and one flat end. The rounded end is used to press the string and the flat end is used to stop the string pressing process. This one rounded and one flat pattern is inspired by MIT. [1]



Fig. 25. Reference of the gear like pluck designed by MIT.



Fig. 26. Gear-like pluck element front view.

Still, the video shown in the MIT page only show case the plucking of the string using the fan-like gear if and only if non of the string is press. Showcasing that the problem of fan not able to finger-pick the string when it is pressed due to the height difference [1]. In our project, same what we mentioned in previous paragraph, the motor will therefore be set with a small gradient in order to be able to press the string even if it is pressed down. Therefore, the plucking element will also tilt forward a bit. To deal with as shown in the bottom view and the side view, the while string pressing area is rounded to prevent the situation of string getting stuck with sharp side and angle.

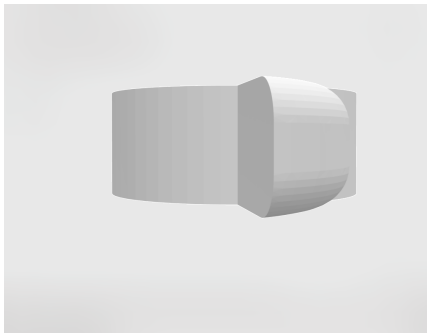


Fig. 27. Gear-like pluck element side view.

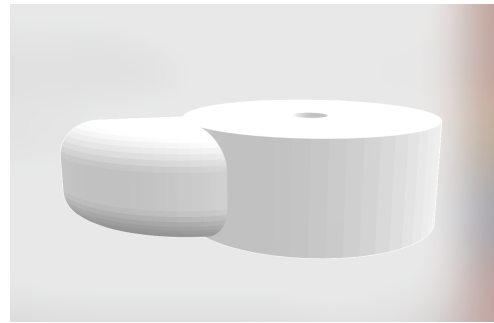


Fig. 28. Gear-like pluck element bottom view.

In the project, to minimize the vibrational affect of the string press, a double layer design is carried out. With such, it can minimize the x-axis width of the rack, hence less affected by the vibration, and enhance the accuracy of fret pressing.

In the top view of both upper slider stand and lower slider stand there are four M5 screw holes for mounting the slider in the right position along with the L-shaped stabilizer for create a stable environment for presenting auto robot playing.

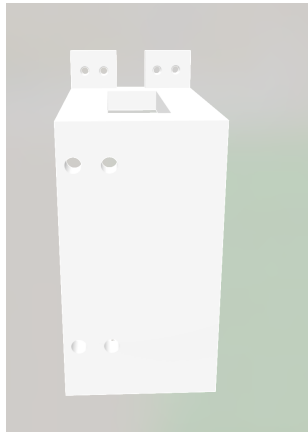


Fig. 29. Double layer top slider holder top view.

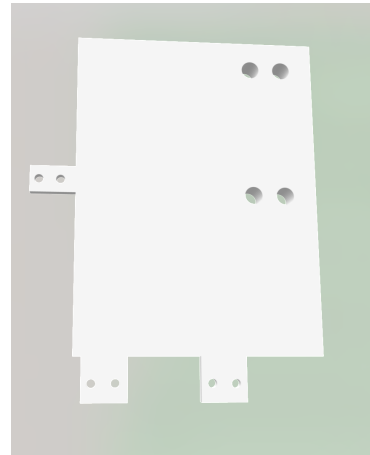


Fig. 30. Double layer bottom slider holder top view.

Similarly, in the side view of both the upper slider stand and the lower slider stand, there are eight M5 screw holes on the small legs to mount the double layer top slider holder on the wooden board. To highlight, the bottom slider holder is L-shaped on one side to reserve space for the bottom level slider's stepper motor.

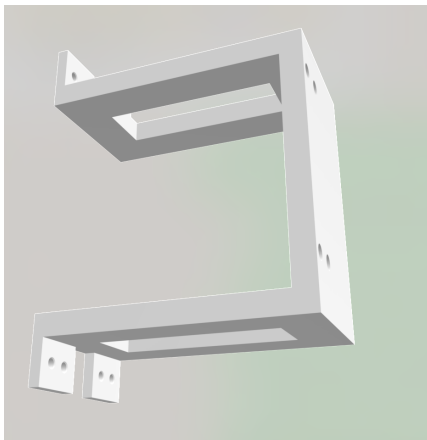


Fig. 31. Double layer top slider holder side view.

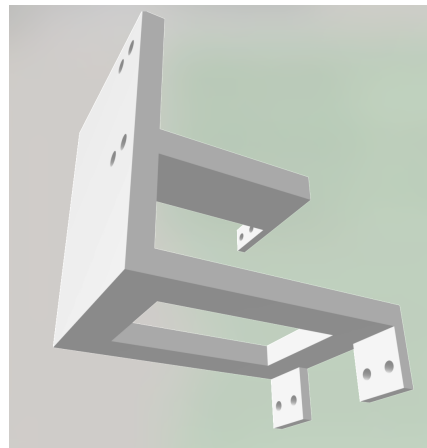


Fig. 32. Double layer bottom slider holder side view.

The bottom view of both holders gives a better picture of how the different M5 screw holes are organized to provide its functionality.

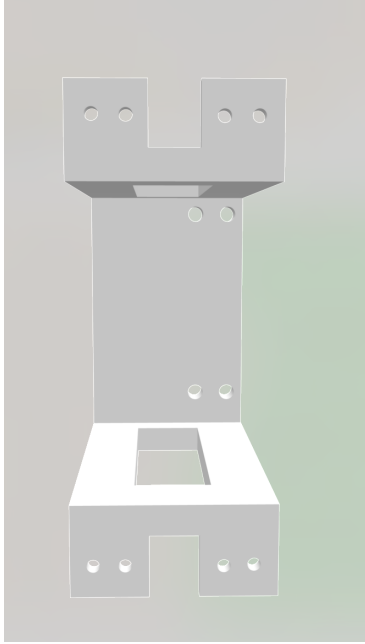


Fig. 33. Double layer top slider holder bottom view.

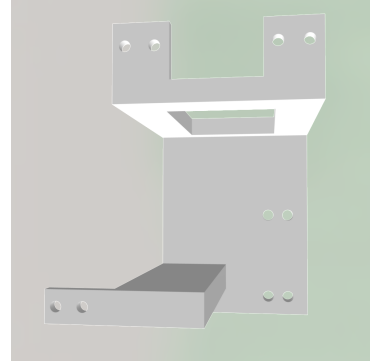


Fig. 34. Double layer bottom slider holder bottom view.

As the robot is systemically mounted along the y-axis, the upper and lower double layer holders for the sliders on the left and right are flipped along the y-axis to match different sides.

Besides the ukulele, stabilizing the slider is also important. The original design of the stand of holder is a triangular shaper stand that support the slider with the tip of the triangular shape. An L-shaped stabilizer is designed to replace all four stands of the original slider. With the screw hole at the bottom part of the L-shaped holder, it allows screws to be installed onto the wooden board. Two separate designs are made to ensure that the stabilizers do not collide with each other.

The design draft is shown below:

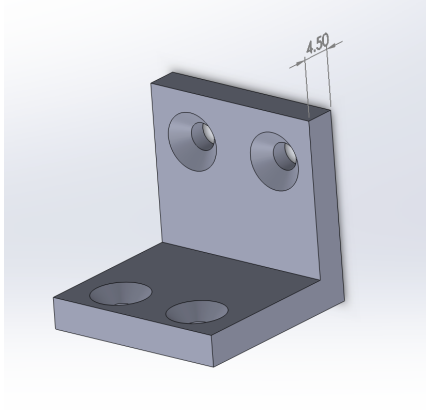


Fig. 35. L-shaped stabilizer.

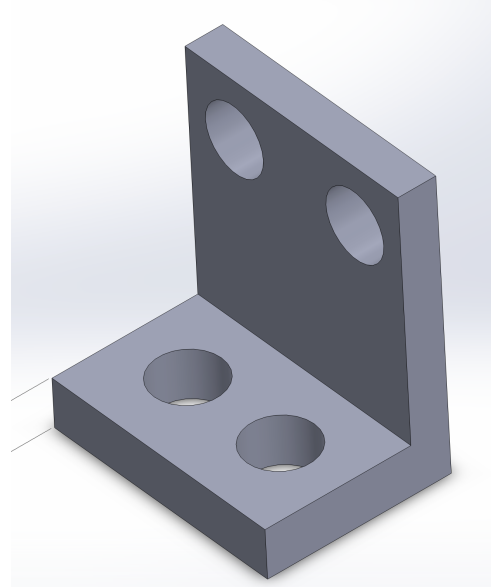


Fig. 36. Shorten version of L-shaped stabilizer to avoid overlapping of components.

## VI. SOFTWARE

### A. Mobile Application

1) *Introduction:* The mobile application part is the only part of software that the user will directly interact with. The main feature of the software part is that the user can simply send command to the server side.

2) *Stack Overview:* The application is written in React Native. We use React Native instead of others like Flutter or native language like Swift. First, it has a relative large community where we can find packages and solutions more easily. Second, it uses React like structure and Typescript, which our team is familiar with. Third, it can compile iOS and Android Version at the same time, which reduce the duplicate work on different platforms. Also, we choose Typescript on top of JavaScript, where we can have less runtime errors as we can discover the mismatch of type on compile.

We have serval packages installed. We have used the following package due to reduce the workload on implementing common features and increase the security level of our code. Here are the important packages that we selected for this project.

1) react-native-ble-plx

It is use to communicate with BLE devices. The reason why we select this package is because it can send the data on a single communication as an array, not bit by bit. We have also considered some BLE Serial Packages, however most of them are outdated and the transmit may be unstable as some bits may be lost. Also, we select BLE over the Bluetooth Classic is because iOS only supports connecting certified Bluetooth Classic devices on App.

2) react-native-document-picker

It allows us to select document directly from device. Hence, we can directly handle the file and upload it to the device server.

3) react-native-sha256

To ensure the accuracy of the file transmitting, we have introduced the checksum mechanism, which we will check if each check and each file is sent correctly via BLE. The checking method is using sha1, where we are using the above package to handle the

calculation.

3) *UI Design:* When the user enters the application, it will first show a view that the user can select the device they want to interact with. It allows the user to control multiple robots if they have. All device discoverable Bluetooth LE (BLE) device will be shown, while the Guitar Robot will be highlighted. After that, it will show a connecting view. When the user is successfully connected. They can interact with the device.

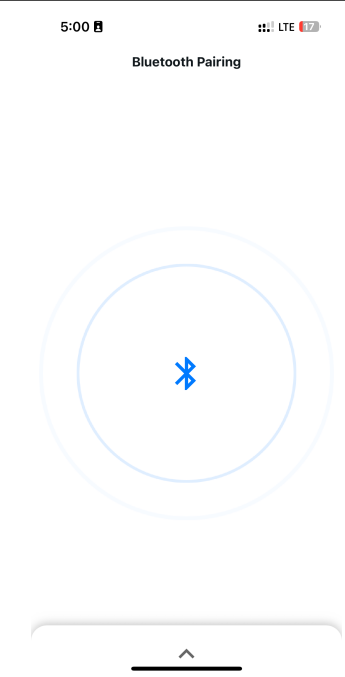
Here are the different pages of the UI.

TABLE II: UI Screenshots

| No. | Image                                                                               | Description                                                                                                                                                                                                                                                          |
|-----|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1   |  | <p>This is the splash screen page where the user will see after they start the app. Before the main screen is loaded, this screen will appear until all required functions are ran successfully. After that, the application will move on to the pairing screen.</p> |

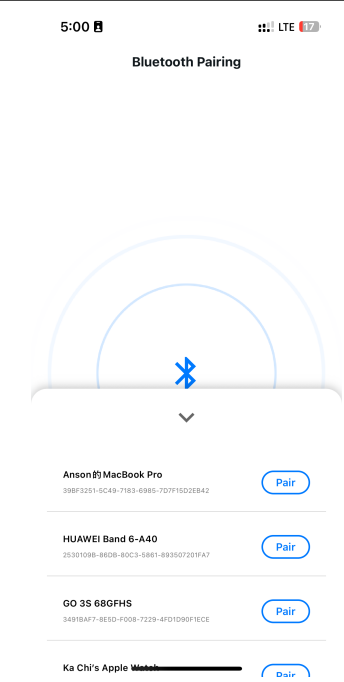
Continued on next page

TABLE II: UI Screenshots (Continued)

| No. | Image                                                                             | Description                                                                                                                                                                                                                                                                                                                                                      |
|-----|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2   |  | <p>The app will now enter a pairing screen. If error are found, it will show an error toast to warn the users. Otherwise, it will keep scanning for Bluetooth devices surrounding.</p> <p>Also, the Bluetooth icon has a ripple effect to have a better UI experience, the user can understand that the program is keep searching devices around the device.</p> |

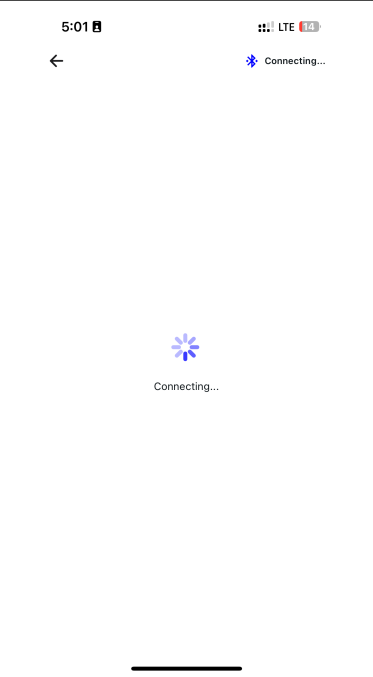
Continued on next page

TABLE II: UI Screenshots (Continued)

| No. | Image                                                                             | Description                                                                                                                                                            |
|-----|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 3   |  | We have decided a drawer in the pairing screen. If the user decided to select their own robot, or there is multiple robot available. They can select it from the list. |

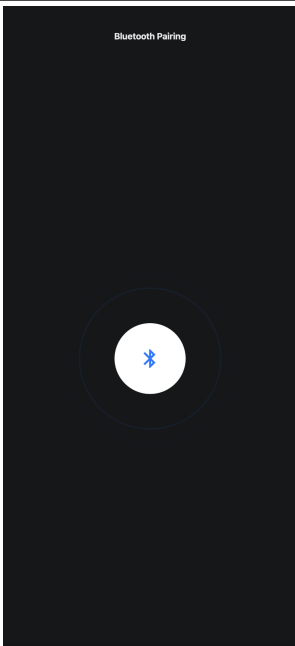
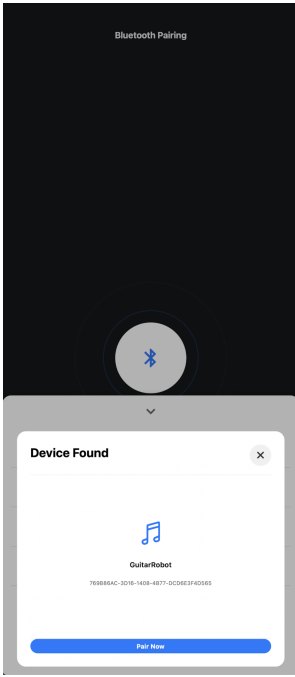
Continued on next page

TABLE II: UI Screenshots (Continued)

| No. | Image                                                                             | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-----|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 4   |  | <p>After the user has selected the device, it will show a connecting page. The user have to wait until the connection is fully established. In this stage, the program will discover all the service and characteristic of the BLE device. After all details are gathered, it will show the next page.</p> <p>If the device is disconnected, the program will also return to the connecting page. That makes the user knows the connection is disconnected. This may prevent the user misunderstand that the device is unresponsive.</p> <p>Besides, to prevent the fluctuation of the connection, we have set up an delay mechanism. Only the connection stables for more than a second will be treated as connected successfully. It may prevent the screen splashing between connecting and the function page.</p> |

Continued on next page

TABLE II: UI Screenshots (Continued)

| No. | Image                                                                               | Description                                                                                                                    |
|-----|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| 5   |    | <p>Our program also supports dark mode to let the user have a better experience at nighttime.</p>                              |
| 6   |  | <p>The program will also match by the BLE name to suggest possible devices, this may let user have a better UX experience.</p> |

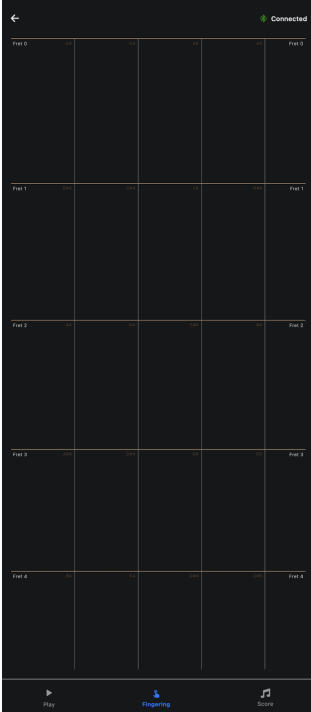
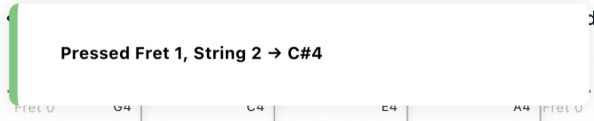
Continued on next page

TABLE II: UI Screenshots (Continued)

| No. | Image | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-----|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7   |       | <p>This is the chord playing page of the application. We uses FlatList to let user can scroll horizontally on the bars to select the note. In this case, the user can intuitively select the notes they desired. After that, the user can click on the buttons to send the chord commands through the BLE service. We have use React.memo to store each lines to prevent the FlatList uses too much memory on heap. Also, useFocusEffect and useEffect are used to handle cases on resize and re-entering the tab.</p> |

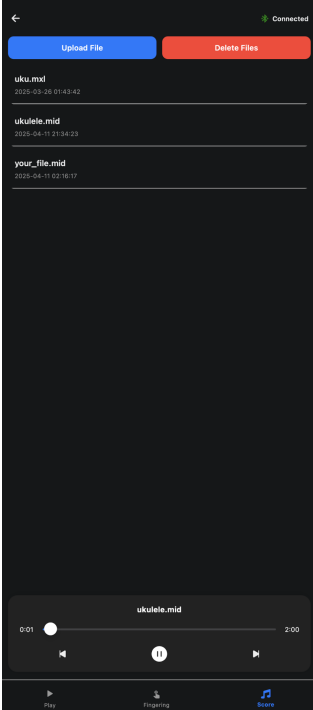
Continued on next page

TABLE II: UI Screenshots (Continued)

| No. | Image                                                                              | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-----|------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 8   |  | <p>This is the fingering page of the application. The page is designed to look like the actual fret of the ukulele. After the page is loaded, it will generate the strings and fret using the scale of the actual ukulele. When the user clicks on the fret, it will send a command using BLE service. If the user wants to play a single note or perform improvisational accompaniment, they can directly click on the line. Animations are also added to each string to let the user know the string is pressed. When the command is successfully run, the string will vibrate and a toast will popup to show the note is playing on the device.</p>  |

Continued on next page

TABLE II: UI Screenshots (Continued)

| No. | Image                                                                              | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-----|------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 9   |  | <p>This is the score playing page of the application. The user can choose to upload, delete or play score. All commands and actions will be sent using BLE service. After each file action are done, the application will retrieve the audio list from the device server again to ensure it is real time. When the user clicks on the audio, the audio player will be pop up and the user is able to adjust the time, play, pause and go to the next or previous song. If the user is going to delete the file, they can click the "Delete Files" button, after that the user can select the files to delete.</p> <p>We have also used useFocusEffect to auto pause the audio. If the user leave this tab, the music will be stop playing to prevent the motor keep moving when user wants to perform another action. Also, serval callbacks are used to communicate with the BLE service while uploading.</p> |

4) *Architecture*: In the application, we have three tabs and two screens currently. They are currently switched by Tabs.Screen and Stack.Navigator.

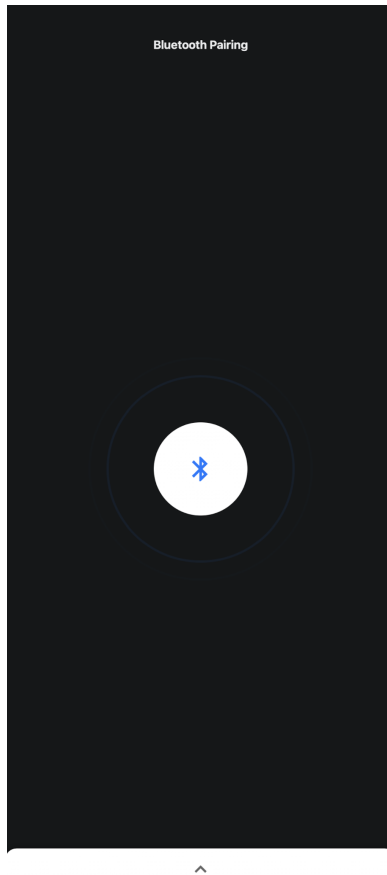


Fig. 37. Before routed in navigator.

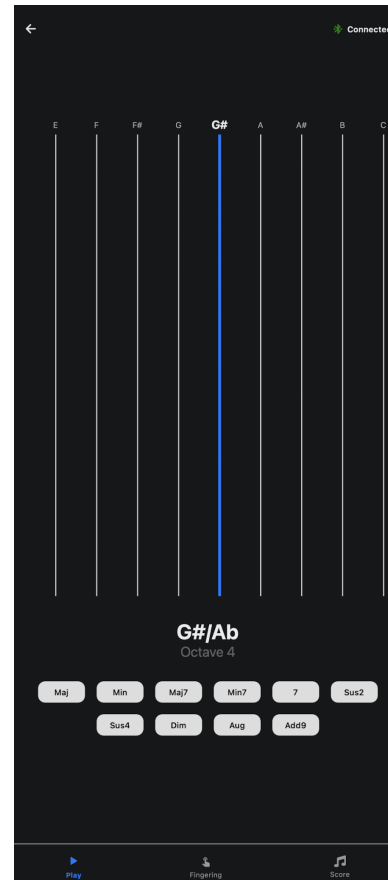


Fig. 38. After routed in navigator.

We use navigator to route the user into device function page. As the user can easily swipe back to the Bluetooth selection page. It makes a clear distinction between the device is selected or not. The user can clearly understand that they are

We use Tabs for navigating between functions. It can preserve the device when we switch between three tabs, to prevent the user need to select device again when they go to another tab. The user can choose different functions using the tab to transfer.

5) *BLE Logic*: We have implemented an Bluetooth service. When the service is first called, it will create a new BleManager Object to the class in the constructor. We placed the BleManager in service instead of UI is because it can prevent create multiple BleManager Object and be

interrupted when we switch between different Views. In different UI of program, we can now easily call the BLEManager functions in the service.

When the program is started, the BLE Service will first discover the signal that is currently broadcasting. Also, it will show devices that are already paired and connected. When the user clicks on it, it will try to connect to the device. After that, it will discover it's characteristics. If there are characteristics returned, the user will be directed to the connected view. After the user send commands, the program will check the corresponding characteristics, then send the command to the that universally unique identifier (UUID). If the server lost connection or no corresponding UUID is found, the server will show an error screen and let the user to reconnect again.

The BLE Service has the following characteristics and functions.

TABLE III: BLE Characteristics and Functions

| Function        | Characteristic | Description                                                                                                    |
|-----------------|----------------|----------------------------------------------------------------------------------------------------------------|
| connectToDevice |                | 1) Connect to the device.<br>2) Discover all services and characteristics<br>3) If error, show the error toast |
| startScan       |                | Start BLE Scanning                                                                                             |
| setDevice       |                | Save the device object to the class to prevent passing though by properties multiple times                     |
| checkConnection |                | Checks the current connection status of BLE                                                                    |

Continued on next page

TABLE III: BLE Characteristics and Functions (Continued)

| Function            | Characteristic | Description                                                                                                                                                                                                                                                                                                          |
|---------------------|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| sendCommandToDevice | MotorWriteChrc | <ol style="list-style-type: none"> <li>1) Encode the command by base64</li> <li>2) Write the command to it's characteristic</li> <li>3) Read the response from the characteristic</li> <li>4) If error, retry up to three times.</li> <li>5) If remains error after three attempts, show the error toast.</li> </ol> |
| getAudioFileList    | ListFilesChrc  | Retrieve the audio list and update the UI                                                                                                                                                                                                                                                                            |
| playAudioFile       | PlayAudioChrc  | Send the play audio command with it's filename and the time-mark.                                                                                                                                                                                                                                                    |
| pauseAudioFile      | PauseAudioChrc | Send the pause audio command                                                                                                                                                                                                                                                                                         |
| deleteAudioFiles    | DeleteFileChrc | Send the delete file command and the filenames to delete.                                                                                                                                                                                                                                                            |

Continued on next page

TABLE III: BLE Characteristics and Functions (Continued)

| Function         | Characteristic | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|------------------|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| sendFileToDevice | FileWriteChrc  | <ol style="list-style-type: none"> <li>1) Retrieve the file from device.</li> <li>2) Detect if it is a MIDI or MusicXML file.</li> <li>3) Turn the file Content to base64 and separate each 20 characters (around 15 bytes) to a chunk.</li> <li>4) Send the file name buffer to device server.</li> <li>5) Process each chunk and calculate the checksum by sha1.</li> <li>6) Send the chunk and the chunk index to the device server.</li> <li>7) Retrieve the checksum calculated by the server side.</li> <li>8) If the checksum is same, update the progress bar to UI and send next chunk.</li> <li>9) If the checksum is different, resend the chunk up to three times.</li> <li>10) After the last chunk is completed, send the "EOF" symbol and wait for the checksum of the full file.</li> <li>11) If the checksum is different, show the error toast.</li> <li>12) If the checksum is same, confirm the file to the server.</li> </ol> |

6) *Deployment:* The application is currently published on App Store and Google Play as beta version. The links are provided in appendix.

## B. Raspberry Pi

1) *Introduction:* The Raspberry Pi program is written in python. The usage of Raspberry Pi is treat as a device server, it trans-code the signal from the mobile application to the Arduino board, and stores the data of the score. We select python as the development language is because it has a large community and there are multiple packages on Bluetooth or UART connections.

2) *Workflow:* The Raspberry Pi will first scan for the device on UART. When the user is connected, they can send commands to the Arduino Mega through the Raspberry Pi.

When the device is setup, it will automatically run a python program every time on start up. It divides into multiple parts to control different components. After that, it will start the Bluetooth service in a thread. Then, it will start receiving user input. Also, a asyncio loop will be started for handling the score.

For the Bluetooth part, the whole part is run in a thread. It can make the terminal is usable, that the user can input in terminal and via Bluetooth in the same time. It will first initialize the Bluetooth service. It will turn on the Bluetooth LE mode and start broadcasting the device. Also, it will change the IO capabilities, which let the device can be paired without entering a pin code. It can make the device connectable without a monitor. After the service is initialized, it will start to advertise itself.

3) *GATT server:* After the Bluetooth server is setup, it will start running a GATT server. Each service has a unique UUID, hence the mobile application can send the command to the specific characteristic.

TABLE IV: BLE Characteristics and Functions

| Service      | Characteristic | Description                                                              |
|--------------|----------------|--------------------------------------------------------------------------|
| MotorService | MotorWriteChrc | 1) Parse the command string<br>2) Send the signal to the specified motor |

Continued on next page

TABLE IV: BLE Characteristics and Functions (Continued)

| Service          | Characteristic | Description                                                                                                                                                                                                |
|------------------|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PlayAudioService | ListFilesChrc  | <ol style="list-style-type: none"> <li>1) Search the score saving directory</li> <li>2) Send all results except the temporary files to the client</li> </ol>                                               |
| PlayAudioService | PlayAudioChrc  | <ol style="list-style-type: none"> <li>1) Retrieve the filename and timestamp to play.</li> <li>2) Run the music scheduler asyncio loop.</li> <li>3) Throw error if the play request is failed.</li> </ol> |
| PlayAudioService | PauseAudioChrc | Pause the music scheduler asyncio loop.                                                                                                                                                                    |
| PlayAudioService | DeleteFileChrc | Send the delete file command and the filenames to delete.                                                                                                                                                  |

Continued on next page

TABLE IV: BLE Characteristics and Functions (Continued)

| Service             | Characteristic | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|---------------------|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FileTransferService | FileWriteChrc  | <ol style="list-style-type: none"> <li>1) Retrieve the filename from client.</li> <li>2) Retrieve each chunk and calculate the checksum by sha1.</li> <li>3) Send the checksum back to the client.</li> <li>4) If the index number is increased, means that the file is successfully processed, then we will save the file from buffer. We will instantly save the file to prevent buffer overflow.</li> <li>5) If the index number is not increased, means that the chunk maybe corrupted and it will overwrite the buffer.</li> <li>6) If the index number restarted from 0, means that the file is resent, it will overwrite the file on disk.</li> <li>7) After the last chunk is completed, the "EOF" symbol will be sent.</li> <li>8) The checksum of the whole file will be sent to client.</li> <li>9) If the checksum is same, the client will confirm the file.</li> <li>10) The server will rename the temporary file to the uploaded file name, hence the user can get it from file list.</li> </ol> |

4) *Command Parsing*: For the user input part, we will parse the input commands into different components. The user can choose to control the specific motor by selecting the Mega board and the motor target. Also, it can directly input the chord or the calibrate command. The user can input via the terminal or the Bluetooth thread. The terminal is for backup usage in case the user

have no Bluetooth device and wants to use the guitar robot. After the command is parsed, it will send to the Mega board by UART Port.

5) *Chord Mapping*: We have implemented a dictionary for the chord mapping task. When the application sent the specific chord command to the Raspberry Pi server, we will map through the array and transform it to a move command. Hence the motors can move to the specific position of the chord.

6) *Score Parsing and Playback Scheduler*: We support MusicXML and MIDI as score format. We have used music21 as a library to transfer MusicXML to MIDI. After that, we will use pretty\_midi to get the details of the MIDI file. We will get the note, octave and it's start and end timestamp. After that, these data will be sent to the asyncio loop.

The asyncio loop is the core of the playing mechanism. When the user decided to play a score, it will load the score using above method. After that, the notes will be played on time in the asyncio loop. The note playing method is similar to the chord playing method. The loop is started as a thread like the Bluetooth instance to prevent blocking and interruption to the whole python program.

7) *Deployment*: We have created an auto setup script for Raspberry Pi. Hence, once a Raspberry Pi with fresh installed software is able to auto setup. The user can switch the Raspberry Pi once it is damaged.

The script enable the serial port on Raspberry Pi, which allows the communication to Arduino Mega. Also, it will download all packages for Bluetooth and python. Then, it will modify the settings of the Bluetooth service, including the broadcast name and discovery period etc. After that, it will setup a service to Auto Run the program, hence the user can run the script on a fresh Raspberry Pi firmware and it will install all dependencies. After reboot, the server should be successfully setup and the user should no longer need to attach the Raspberry Pi to a screen.

### C. Arduino

1) *Introduction*: The Arduino Mega part is written by C++.

When the motor driver starts up, we will calibrate each motor. For example, we will move the motor to the end to ensure it has the same position every time we start up. We will use an end switch to detect its position. After that, it will turn into discovery mode and let the others send commands to it. It will be able to receive command through the UART port. We have also normalized the input format to make instructions more readable, details is included in System Design.

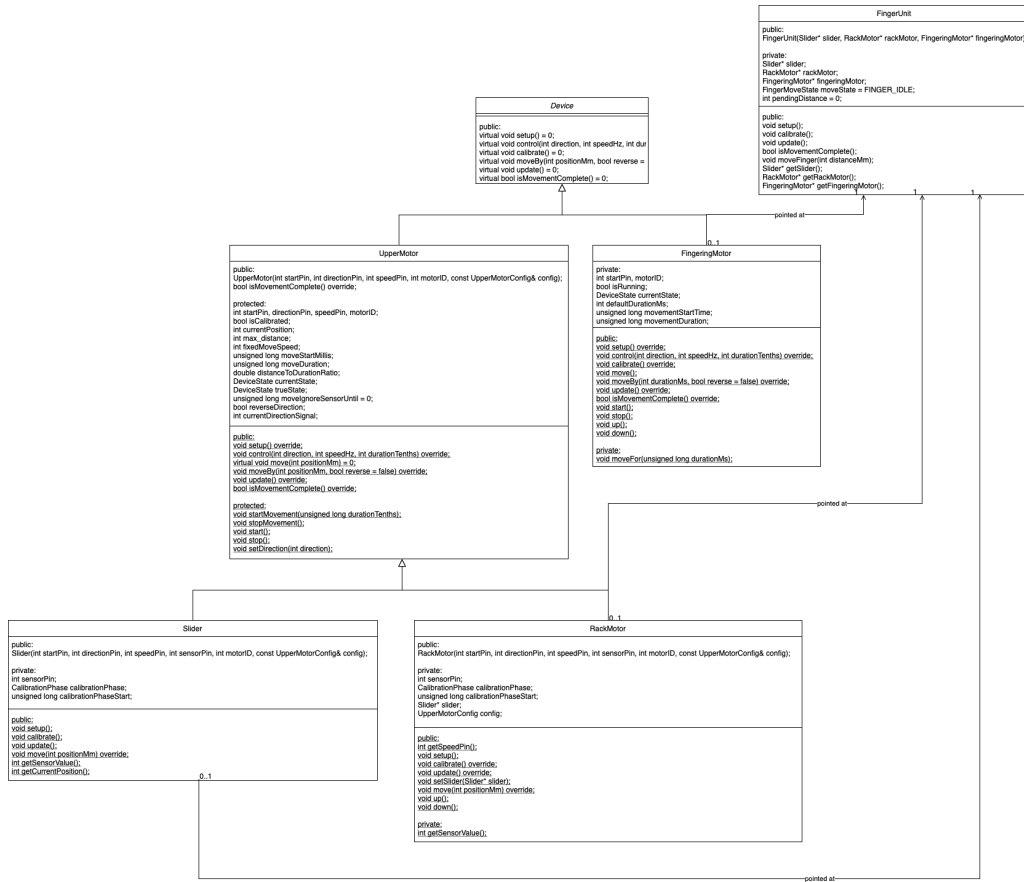
The Mega board will send digital signal to the start pin and direction pin; also it will send analog PWM signal to the speed pin. Hence, we can control the speed, direction and status of the motor by Mega board. Also, the board will keep tracing how much distance did the motor go forwards and backwards for the chord function.

2) *Component Class*: In our program, each component are treated as an object. The object that the main function can directly call is FingerUnit, which is a class that contains the pointer of Slider object, Rack Motor object and Finger Pick Motor object. For every movement, we can directly call the functions like moveFinger() to move the three motors in order. In this case, we can make the code more readable, modularize and organizable.

For the slider and rack motor object, it is inherited by the class UpperMotor. It is because they are both servo motor, hence they have similar properties like startPin, directionPin and speedPin. When we have to control it's movement, we can directly call the functions from their super class to control those pins. Hence, we can reduce duplicate code and make it more readable. When the object needs to modify the functions, we can just directly override the superclass function.

The above UpperMotor class and the Finger Pick Motor is a sub-class of Device class. As the Device class specifies the command that is required, like setup(), calibrate(), to prevent error when the main class needs to call it.

The diagram showing their relationship is as below.



3) *Config*: As different motors may have slightly different on the speed. To reduce the difference between motors, we have introduced a config per motor, where we can choose to modify its property using the above class method. Our properties to change including maxDistance, distanceToDurationRatio, etc.

4) *Finite State Machine*: We have use FSM as in the loop part of the Arduino. We are preventing to use delay functions to prevent freezing. As Arduino board does not support threading, hence we use FSM to make the loop non-blocking. The states are as below:

- 1) Calibration: {CALIBRATION\_INIT, CALIBRATION\_BACK\_OFF, CALIBRATION\_WAIT\_RELEASE, CALIBRATION\_WAIT\_1, CALIBRATION\_SEEK\_SENSOR, CALIBRATION\_DONE}

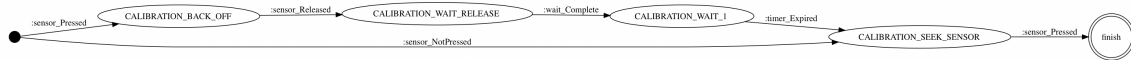
The action are as below:



- 2) Fingering Unit: {FINGER\_IDLE, FINGER\_UP, FINGER\_SLIDE, FINGER\_DOWN, FINGER\_PRESS, CALIBRATING\_SLIDER, CALIBRATING\_RACK,

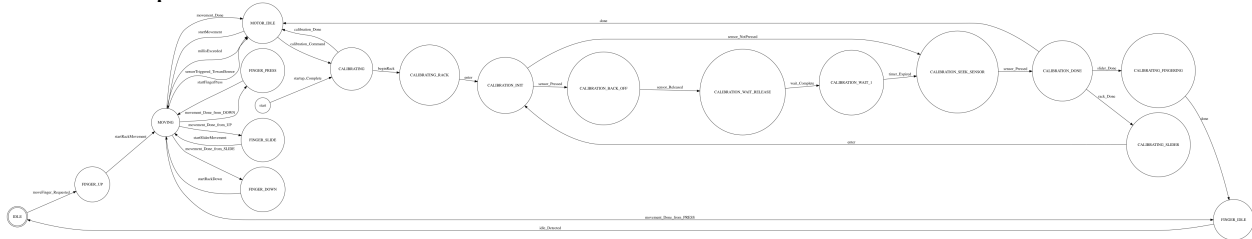
## CALIBRATING\_FINGERING}

The action are as below:



### 3) Device: {IDLE, MOVING, CALIBRATING}

The whole picture of FSM are as below.



The detail operation will be explained below.

### 5) Workflow: Here is the workflow of the Arduino program.

#### 1) Setup:

When the program loads, it will enter a start up function. It will first initialize the Serial communication. After that, it will create objects of FingerUnit and set their own slider, rack motor and finger pick address to its class variable. Then, it will run the setup function of the FingerUnit unit, which means running the setup process of below objects respectively.

- Slider and Rack Motor: Set the start, direction and speed pins to its class variable.
- Finger Pick: Set the start pin to its class variable.

After that, it will run the calibration function of the object. However, the process of calibration is running one by one to prevent collision of each motor. For example, if the rack motor is pressing down when the slider is moving, it may be damage to the rack as it may drag the rack when it is pressing the string. In the setup part, we have just run the calibrate function on the rackMotor first. Then we change the FSM of FingerUnit to CALIBRATING\_RACK. After that, we setup process of arduino is ended, and it started to load the loop process. The calibration function of each object is as below.

- Slider and Rack Motor: Set the FSM to CALIBRATION\_INIT mode.
- Finger Pick: Move the motor for 2000 second.

#### 2) Loop:

In the whole loop, the Arduino board will first update each fingering unit, which contains slider, rack motor and finger pick motor. After that, it will check if the serial port has input. If there is a command send from the UART port, it will process the command.

Now, as only the rack motor is set to CALIBRATING mode, hence only the rack will have action. For simplicity for explanation, we will assume the calibration of rack motor is completed. The details of each FSM including calibration and action will be explained later. The loop function will detect if the calibration is completed by the isMovementComplete() function in the rack motor class. If it returns true, the state will turn to CALIBRATING\_SLIDER, and eventually it will calibrate the fingering pick motor. After all these action, the state will become FINGER\_IDLE.

The case of moveFinger is similar. For the whole process of one note, it will first make the rack motor up, to prevent collision of the string and the rack. Then, it will move the slider to the corresponding position. After that, it will put the rack motor down to press the string.

For the FSM setup, it will first move the rackMotor up, and set the state to FINGER\_UP. After that, when the movement is completed, it will turn to move the slider, then put the finger down and press the finger down. Finally, the state will become idle.

As all of the above cases uses conditional operator like "if" and "switch", the program are non-blocking. Hence, it can handle the queue process easily.

### 3) Each component:

Here are the each update function and FSM of each component.

- a) Slider: The FSM is mostly on the CALIBRATING job. In the update function, it will three parts.

- i) Collision detection:

If the slider is not in calibration mode, it will stop the motor from moving if it touches the sensor. Also, as the sensor represent the position 0, it will also update the current position to auto calibrate when the user clicks on the first note.

- ii) Moving state:

This will call UpperMotor update function, the super class of slider object. In this function, it will keep checking if the moving action is expired by calculating the moveDuration and moveStartMillis. Hence, it will act as a timer to count when

to stop the slider.

iii) Calibration:

If the motor is in calibration mode, it will enter this condition. In the CALIBRATION\_INIT state, it will detect if it is touched the sensor already. If it is the case, it will turn to back off state. The motor will go back and go to CALIBRATION\_WAIT\_RELEASE state, it will check if the motor has left the motor. After that, it will enter CALIBRATION\_WAIT\_1 state, which lets the motor continue move further for some extra time. After that, it will enter CALIBRATION\_SEEK\_SENSOR state. If the motor has not touched the sensor in the beginning, it will directly enter this state. In this state, the board will keep powering the motor until it pushes the sensor button. After the button is pressed, it will enter CALIBRATION\_DONE state and it will stop moving. After that, it will exit the calibration mode and return to IDLE state.

b) Rack Motor: Same as slider.

c) Finger Pick: No FSM is needed as it has no queue job.

## VII. DIVISION OF LABOR

### A. Detail Work

| Task                                             | Anson | Crystal |
|--------------------------------------------------|-------|---------|
| Hardware Circuit Design                          | 30%   | 70%     |
| Mounting component on wooden board               | 15%   | 85%     |
| Wiring Setup                                     | 20%   | 80%     |
| Component Selection                              | 20%   | 80%     |
| 3D Model Design (Solidwork)                      | 25%   | 75%     |
| 3D Printing                                      | 15%   | 85%     |
| Motor Control Logic (Arduino FSM)                | 70%   | 30%     |
| Fingering Slider Coding                          | 80%   | 20%     |
| Config + Finite State Machine                    | 80%   | 20%     |
| BLE Protocol Design                              | 50%   | 50%     |
| GATT Server (Raspberry Pi)                       | 80%   | 20%     |
| Command Parsing (Raspberry Pi)                   | 50%   | 50%     |
| Fingering and Chord Mapping Logic (Raspberry Pi) | 75%   | 25%     |
| Score Parser, Playback Scheduler (Raspberry Pi)  | 80%   | 20%     |
| Mobile UI Design                                 | 20%   | 80%     |
| UI Implementation (React Native)                 | 60%   | 40%     |
| BLE and GATT Integration (React Native)          | 60%   | 40%     |
| App Deployment                                   | 80%   | 20%     |

### B. Personal Contribution

My contribution is mainly on the design and implementation on the hardware side. The hardware side can be further divided into three sessions, namely, wiring, electronic components and 3D printing.

As a computer science student with no prior experience in any hardware design, I spend a lot of effort on learning new knowledge of wiring and 3D printing. As I understand that it is essential to design and assemble all the hardware component to preform what we have program, both parts need to collaborate seamlessly to carry out our expected outcome, of playing a song

on its own.

Throughout this academic year, I am responsible for hardware design including the research of similar projects done by different parties and understand the pros and cons of different approaches. Hence, the electronic component selections based on the targeted functionality, by writing Arduino program to test and designing prototype to see if the component meets our needs and can be used in our project. Hence, writing codes for Arduino to control the logic of the motor.

After checking the basic functionality of the selected components after several times then work on the part for the 3D component design through solidwork and fine-tuning the draft, the positioning and set-up of 3D printing to work. This is an iterative and continuous process throughout the year. As time passes, different components may wear out and always encountered some unexpected error or malfunctionality of hardware. Undergo problem-shooting and ease the problem by fixing and reorganizing the wire to minimize the amount of wire used in a safe way, for example, interconnecting the ground and VCC of all motor driver making it better to organize and only require to power on one power source for whole robot.

Meanwhile, I also participated in setting up Bluetooth connection between Arduino and Raspberry Pi. I also take part in mobile UI design and the UI implementation using React Native. I also mounted the component on the wooden board and soldered wire for fixing the connection and enhance stability.

## VIII. CONCLUSION

### A. Summary

In the current project, we have built a music robot using techniques on hardware and software.

On the hardware side, we have created 3D Models to assemble all electronic components into a robot. After that, we have designed the circuit to power up the motors and handle the signals between Arduino and Raspberry Pi. After the hardware is well set, we also have to handle the software side. Not only providing a great experience user interface for user, but also making communication between the client and the hardware parts.

In this project, we have involved fields on signaling, electronic, 3D printing on hardware side. On software, we also handled Bluetooth protocol, mobile application development, asynchronous programming using Threading and Finite State, and also application deployment.

With combining these work together, we believe we created a seamless robotic music experience for the user.

### B. Future Work

It is believed that this project is just building a foundation for the developing a ukulele robot in CUHK. Meanwhile, based on the lesson we have learned in the trial and error session, we ended up we these hardware component selection and 3D design. We believe that there is still great potential to integrate this project with AI scoring detection. Hoping that in the upcoming future, the user can scan the physical ukulele score with their mobile application and integrate AI for image identification and automatically transform it to electronic scoring format and perform real-time performance.

In addition to that, functionality that supports real-time composition can also be developed to meet our objective of allowing a the robot to work as a ukulele assistant that accompany you with the learning, playing, composing and listening to ukulele music.

Other than software, it is hoped that the hardware design of the robot can be further simplified and make it more portable by reduce in physical size. Or in another perspective, to allow the robot to be used for all kinds of ukulele, instead of the same model all the time, providing higher

flexibility. Moreover, find another motor that is smaller in size but with similar functionality or even the ability to control speed precisely without making significant noise that will affect the quality of the performance.

## REFERENCES

- [1] Leigh, S. (2019). Guitar Machine – MIT Media Lab. MIT Media Lab. <https://www.media.mit.edu/posts/guitar-machine/>
- [2] Luo, C., Tang, P., Ma, Y., Huang, D. (2024). Learning to Play Guitar with Robotic Hands. In Computer Graphics Forum (p. e15166).
- [3] Kapur, A. (2005, September). A history of robotic musical instruments. In ICMC (Vol. 10, No. 1.88, p. 4599).
- [4] Kodama, K., Koutaki, G. (2019, October). Development of guitar playing robot by PWM control of solenoid. In 2019 IEEE 8th Global Conference on Consumer Electronics (GCCE) (pp. 291-293). IEEE.
- [5] zeroshot. "I Built a Robot Ukulele (It Plays Itself)." YouTube, 16 Aug. 2024, [www.youtube.com/watch?v=2MbfsqCwU2Q](http://www.youtube.com/watch?v=2MbfsqCwU2Q). Accessed 26 Nov. 2024.
- [6] ""Robot Ukulele" Mikołaj Wybacz." [www.youtube.com](http://www.youtube.com), 16 Mar. 2021, [www.youtube.com/watch?v=euGtVDGpUWc](http://www.youtube.com/watch?v=euGtVDGpUWc).
- [7] Davison, Timothy, et al. "Mechalele (Self-Playing Ukulele) Repository Citation." 2016.
- [8] "Pirates of the Caribbean(Guitar-Robot)." [www.youtube.com](http://www.youtube.com), 5 Nov. 2019, [www.youtube.com/watch?v=n\\_6JTLh5P6E](http://www.youtube.com/watch?v=n_6JTLh5P6E).
- [9] Athani, V. V. (1997). Stepper motors: fundamentals, applications and design. New Age International.
- [10] Shahrubudin, N., Lee, T. C., Ramlan, R. J. P. M. (2019). An overview on 3D printing technology: Technological, materials, and applications. *Procedia manufacturing*, 35, 1286-1296.

## IX. APPENDIX

*A. Application Download Link*

| Platform | Links                                                                                                                                                                                                                                                                                                                                                         |
|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| iOS      | <a href="https://testflight.apple.com/join/FN3Dxs6W">https://testflight.apple.com/join/FN3Dxs6W</a>                                                                                                                                                                                                                                                           |
| Android  | <ol style="list-style-type: none"><li>1) Login google account.</li><li>2) Join the group on <a href="https://groups.google.com/g/ukulelerobot">https://groups.google.com/g/ukulelerobot</a></li><li>3) Join the beta on <a href="https://play.google.com/apps/testing/com.robot.ukulele">https://play.google.com/apps/testing/com.robot.ukulele</a></li></ol> |